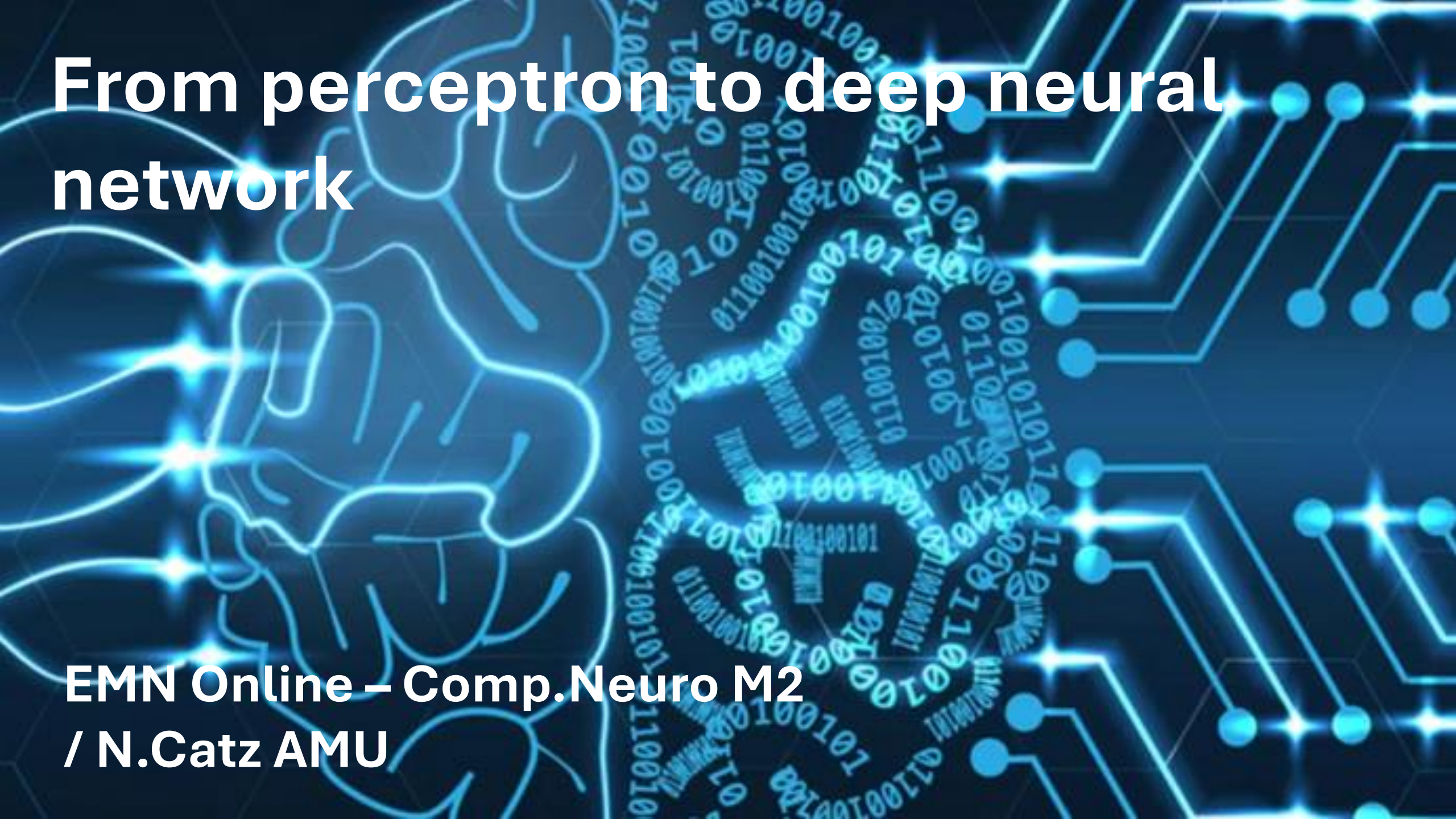
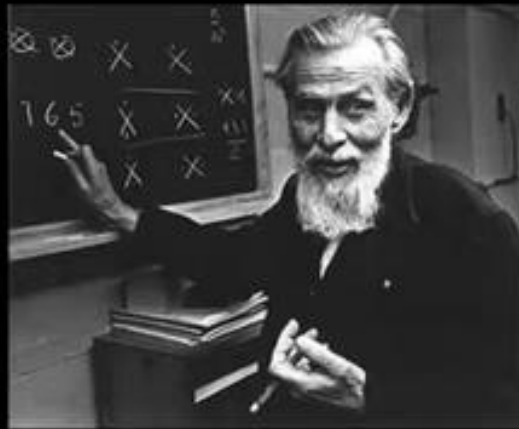


From perceptron to deep neural network



EMN Online – Comp.Neuro M2
/ N.Catz AMU

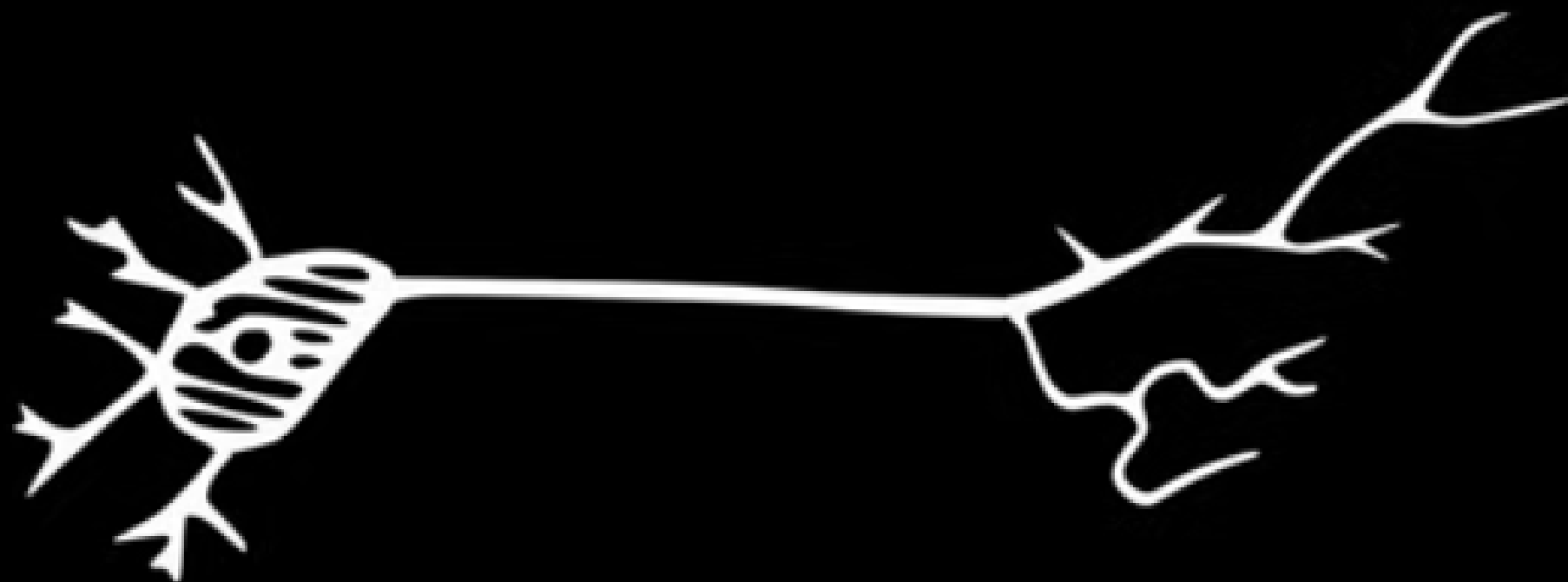
1943 : McCulloch and Pitts
The first artificial neuron



Warren McCulloch

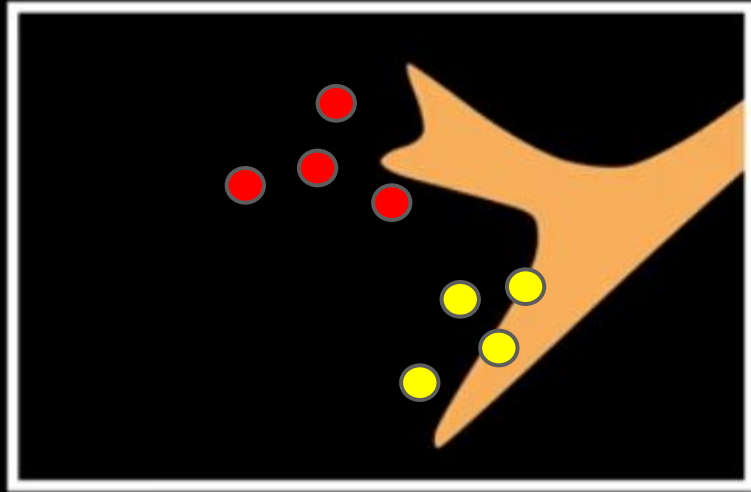


Walter Pitts



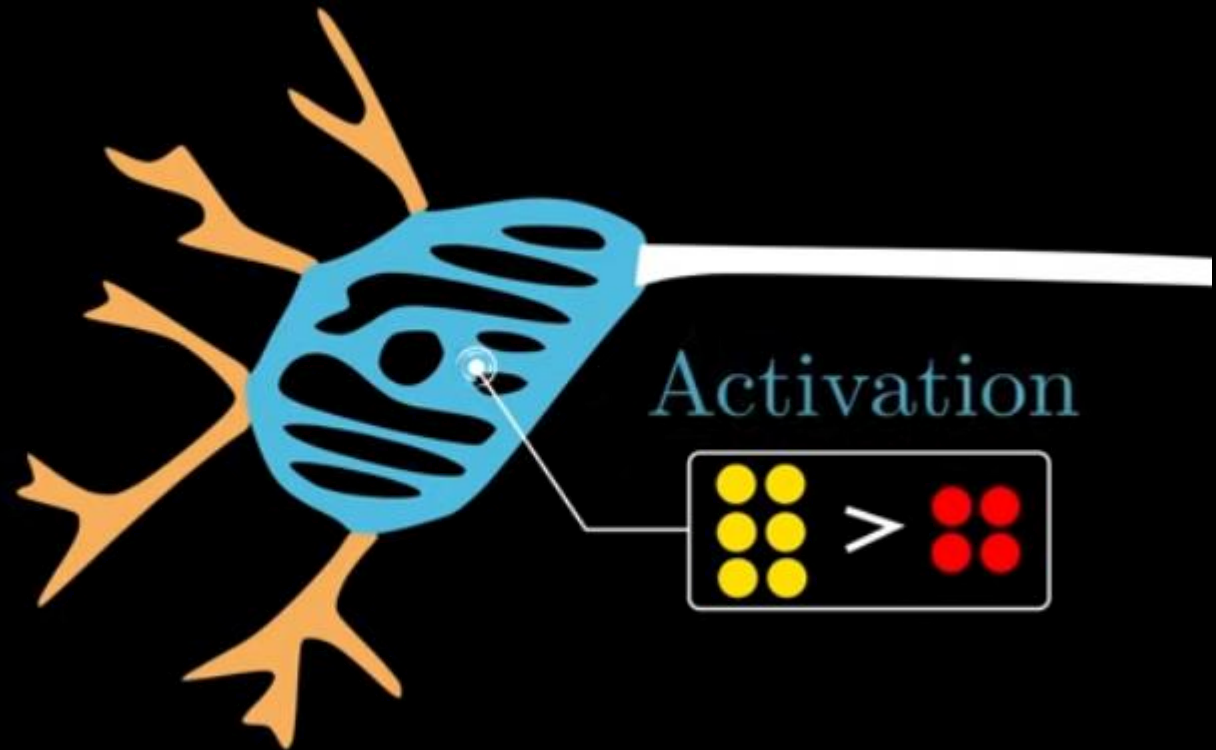
Excitatory : +1

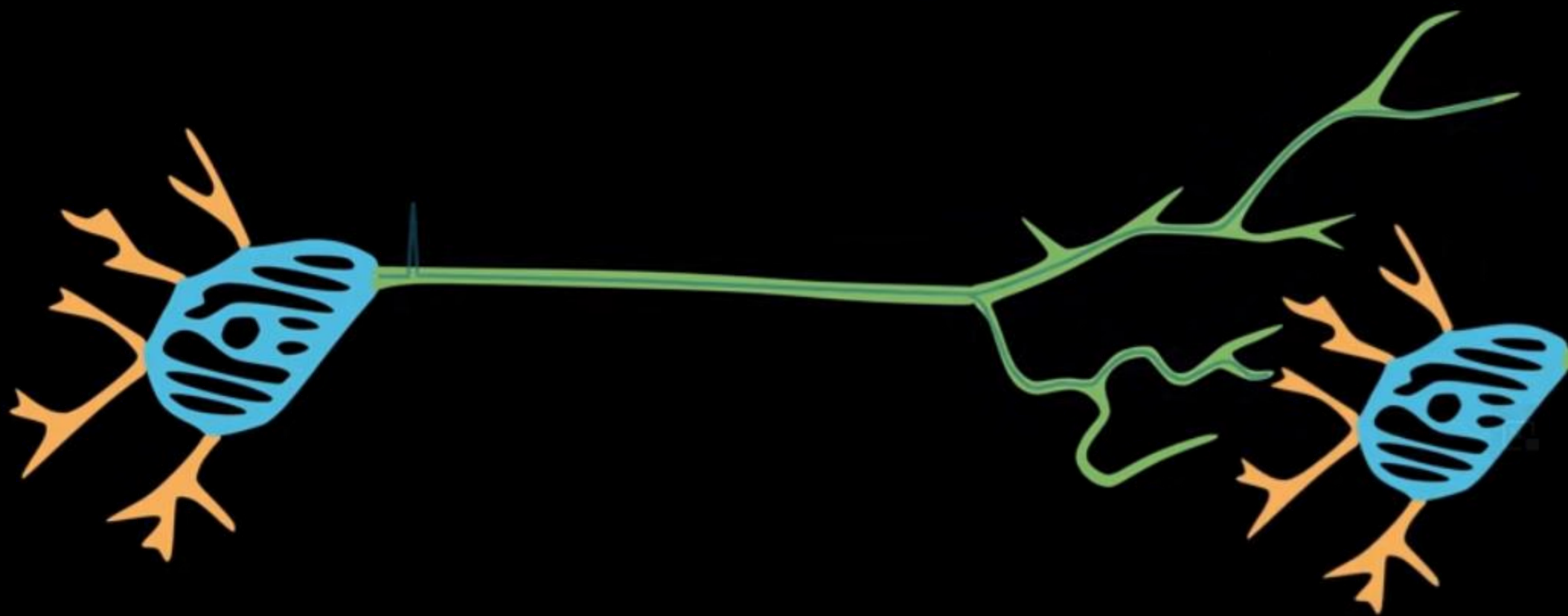
Inhibitory : -1



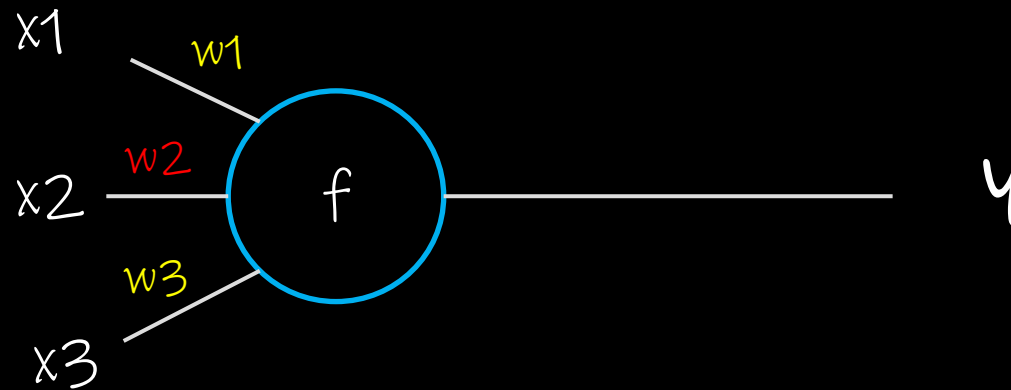
Synapse



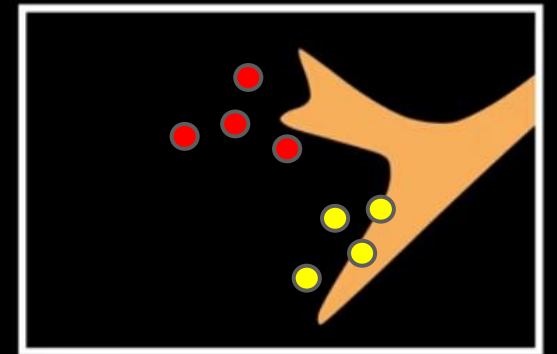




Agregation : $f = w_1x_1 + w_2x_2 + w_3x_3$

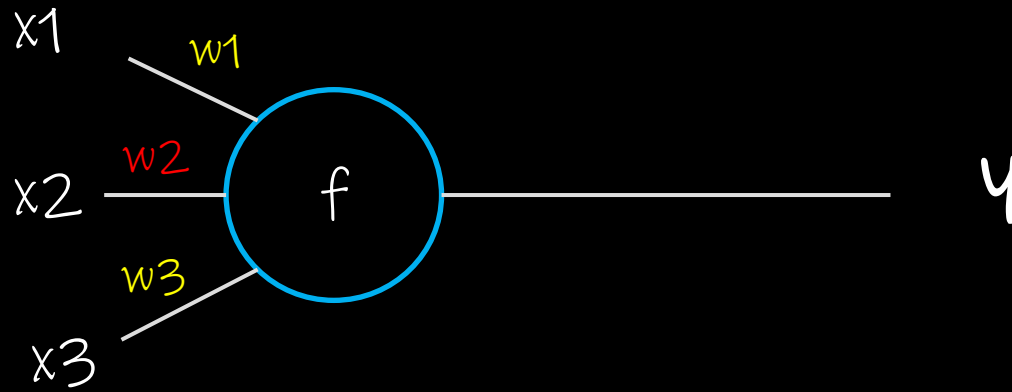


Excitatory : +1
Inhibitory : -1



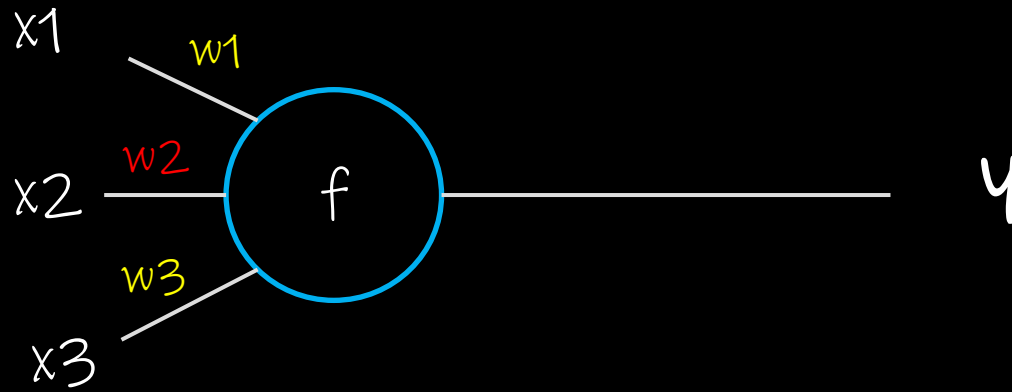
Agregation : $f = w_1x_1 + w_2x_2 + w_3x_3$

Activation :
$$\begin{cases} y = 1 & \text{if } f > 0 \\ y = 0 & \text{else} \end{cases}$$



Agregation : $f = w_1x_1 + w_2x_2 + w_3x_3$

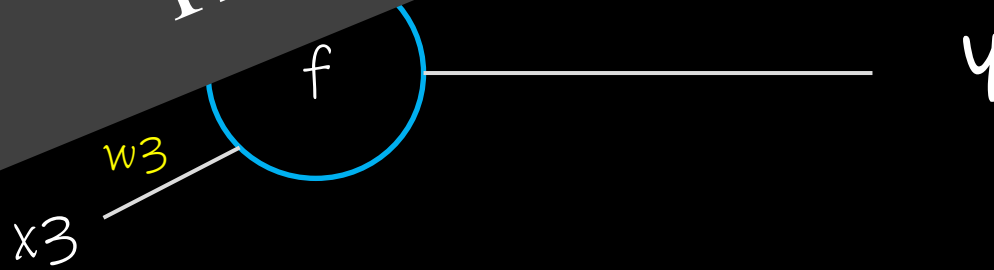
Activation :
$$\begin{cases} y = 1 & \text{if } f > 0 \\ y = 0 & \text{else} \end{cases}$$



Aggregation: $f = w_1x_1 + w_2x_2 + w_3x_3$

Activation:
$$\begin{cases} y = 1 & \text{if } f > 0 \\ y = 0 & \text{if } f \leq 0 \end{cases}$$

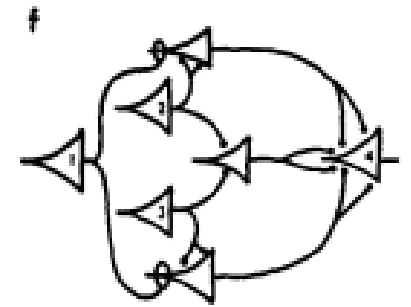
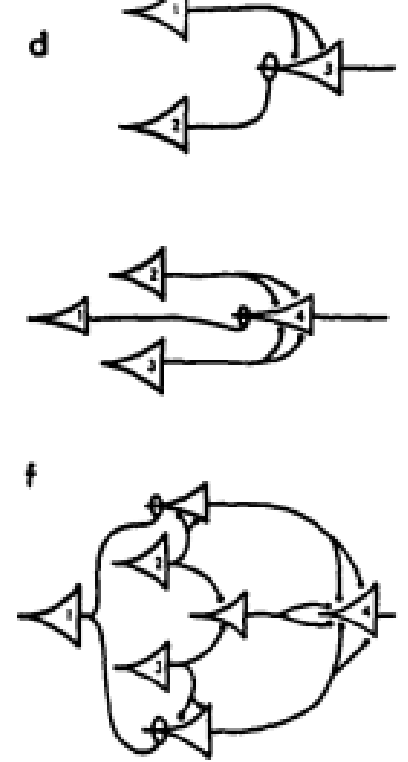
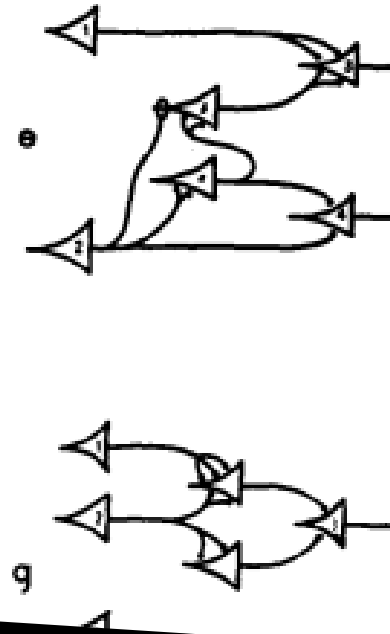
McCulloch and Pitts:
The Threshold Logic Unit



A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY

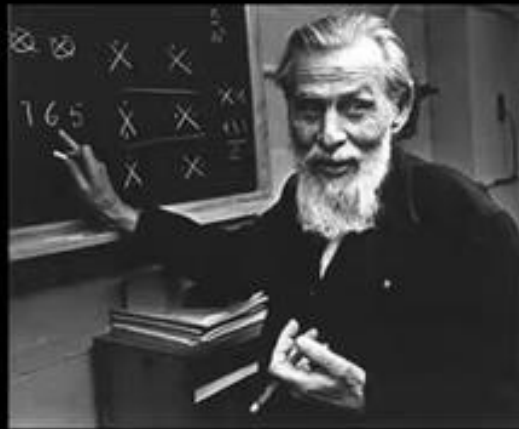
WARREN S. MCCULLOCH and WALTER H. PITTS

Because of the "all-or-none" character of nervous activity, neural events and the relations among them can be treated by means of propositional logic. It is found that the behavior of every net can be described in these terms, with the addition of more complicated logical means for nets containing circles; and that for any logical expression satisfying certain conditions, one can find a net behaving in the fashion it describes. It is shown that many particular choices among possible neurophysiological assumptions are equivalent, in the sense that for every net behaving under one assumption, there exists another net which behaves under the other and gives the same results, although perhaps not in the same time. Various applications of the method



McCulloch and Pitts :
The Threshold Logic Unit

1943 : McCulloch and Pitts
The first artificial neuron

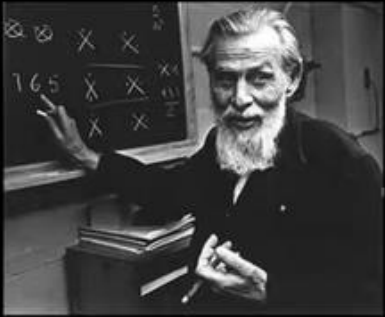


Warren McCulloch



Walter Pitts

1943 : McCulloch and Pitts
The first artificial neuron



Warren McCulloch



Walter Pitts

1949 : Donald Hebb
'The Organization of Behavior',
introducing the concept that
would become Hebbian
learning: "Neurons that fire
together, wire together."



Donald Hebb

1950 : Alan Turing
Computing machinery and
Intelligence

1950 : Alan Turing

Computing machinery and
Intelligence

1949 : Donald Hebb

'The Organization of Behavior',
introducing the concept that
would become Hebbian
learning: "Neurons that fire
together, wire together."



Donald Hebb

1957 : F. Rosenblatt

Invention of the perceptron



Frank Rosenblatt

The perceptron (1957)



Frank Rosenblatt

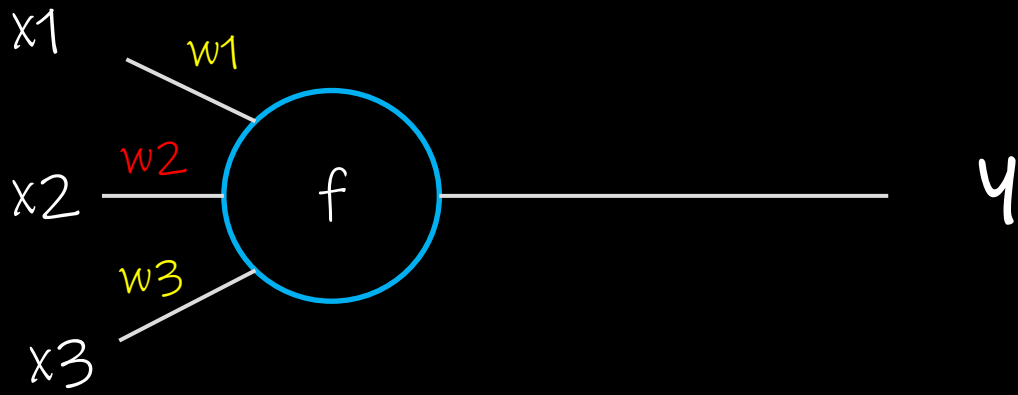
A perceptron is the simplest form of an artificial neural network, designed to mimic how a biological neuron works:

- Takes multiple **weighted inputs**
- Applies a **threshold function** (activation)
- Produces a **binary output** (0 or 1)
- Can **learn** from training data

The perceptron (1957)

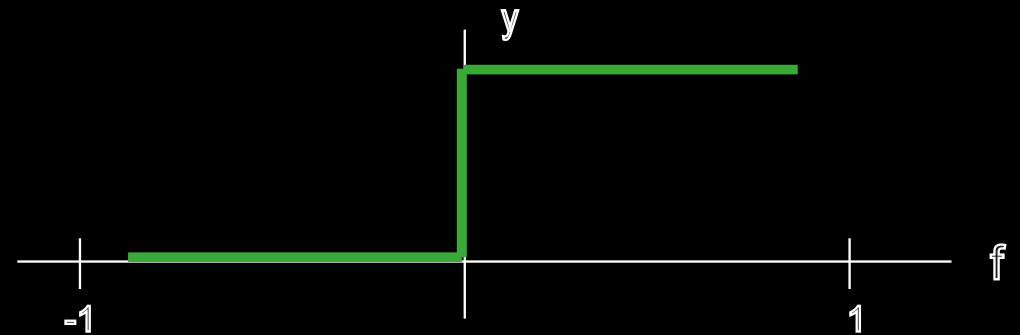
$$f = w_1x_1 + w_2x_2 + w_3x_3$$

$$\begin{cases} y = 1 & \text{if } f > 0 \\ y = 0 & \text{else} \end{cases}$$



Frank Rosenblatt

+ Learning Algorithm



Hebbian theory

When two biological neurons are activated simultaneously, they strengthen their synaptic connections.



Donald Hebb

Hebbian theory

When two biological neurons are activated simultaneously, they strengthen their synaptic connections.



Donald Hebb

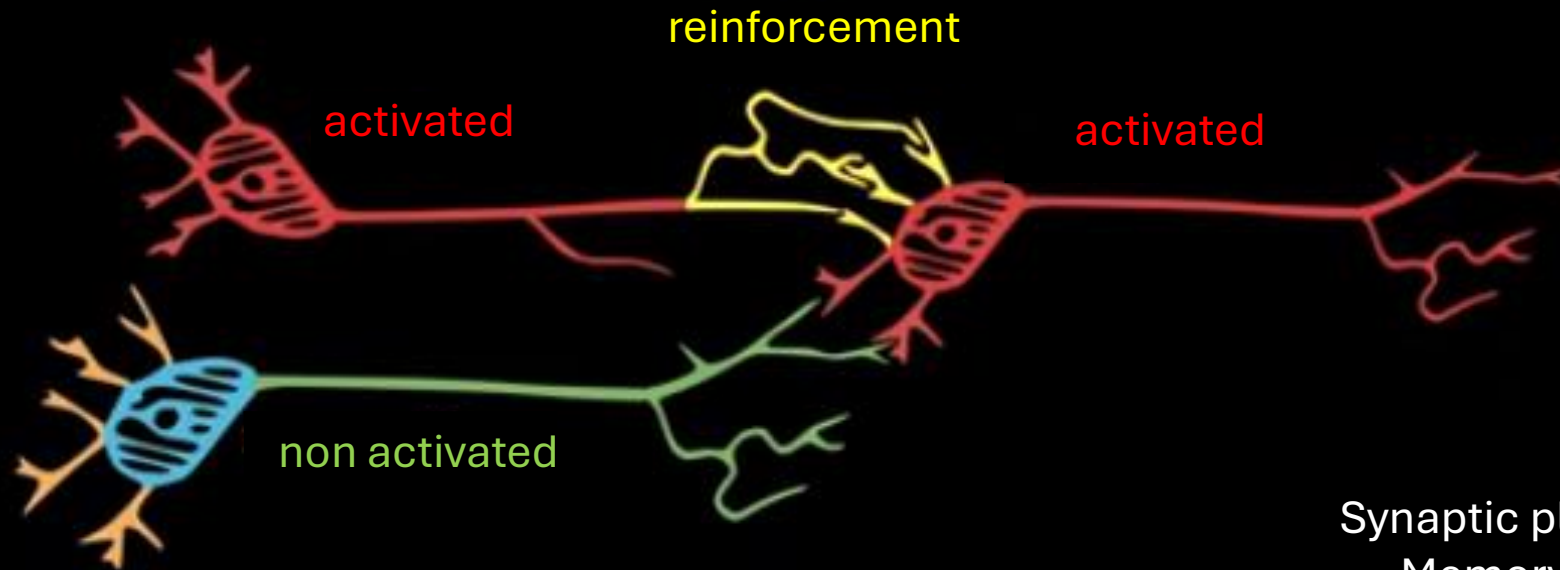


Hebbian theory

When two biological neurons are activated simultaneously, they strengthen their synaptic connections.



Donald Hebb

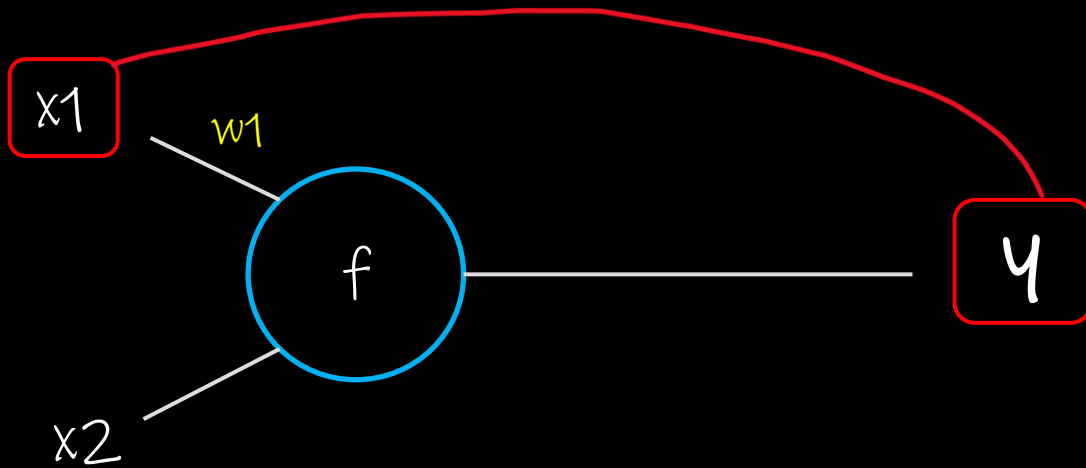


Synaptic plasticity

- Memory
- Learning new skills
- Make new associations

The perceptron (1957)

Train an artificial neuron on **reference data** (X, y) so that it **reinforces** its **parameters** w **each time an X input is activated** at the same time as the **output** y present in this data.



$$W = W + \alpha (y_{true} - y) X$$

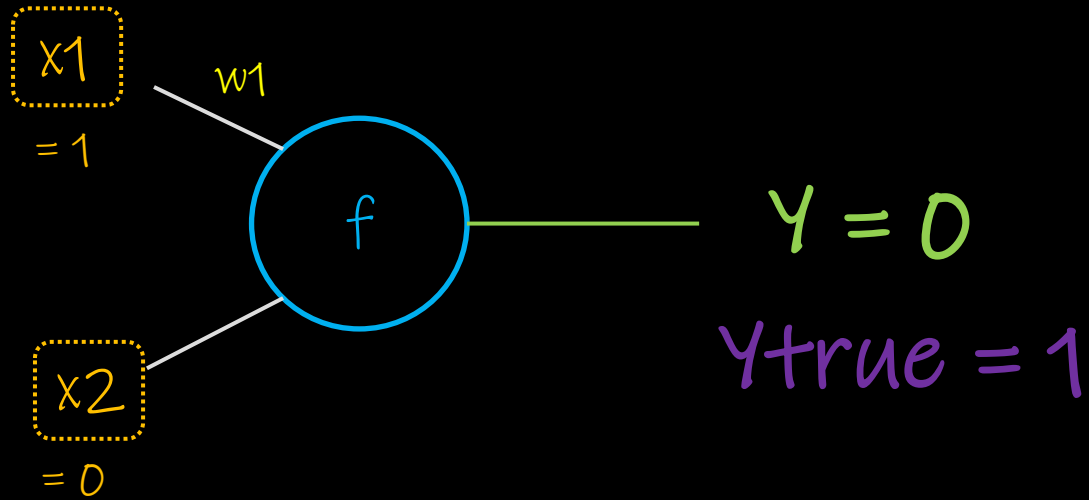
Y_{true} : reference output

Y : produced output

X : input

α : learning rate

The perceptron (1957)



$$W = W + \alpha (y_{true} - y) X$$

$$W = W + \alpha (1 - 0) X$$

$$W = W + \alpha X$$

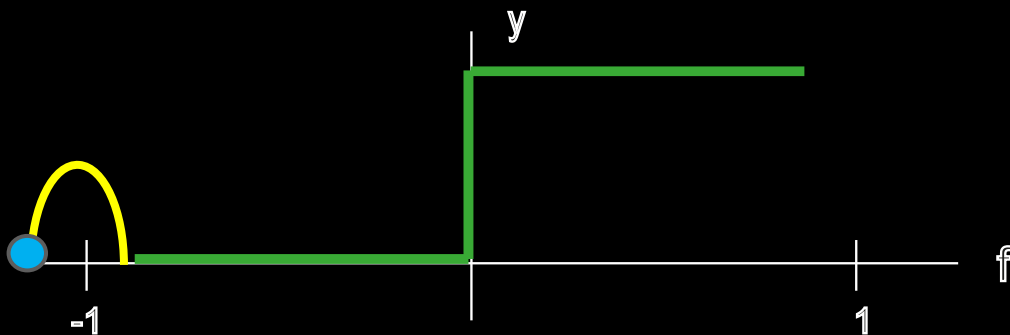
$$f = w_1 x_1 + w_2 x_2$$

$$w_1 = w_1 + \alpha$$

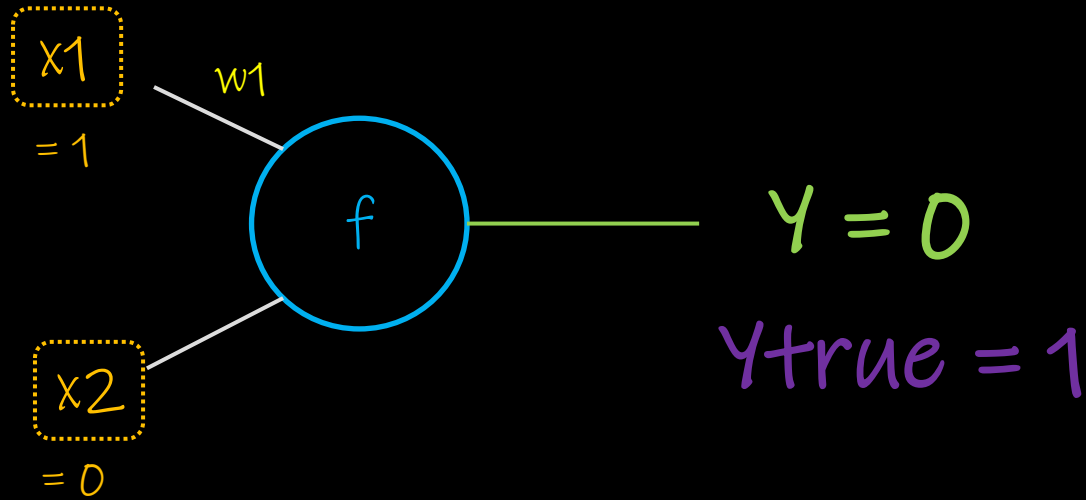
$$w_2 = w_2$$

$$(x_1 = 1)$$

$$(x_2 = 0)$$



The perceptron (1957)



$$W = W + \alpha (y_{true} - y) X$$

$$W = W + \alpha (1 - 0) X$$

$$W = W + \alpha X$$

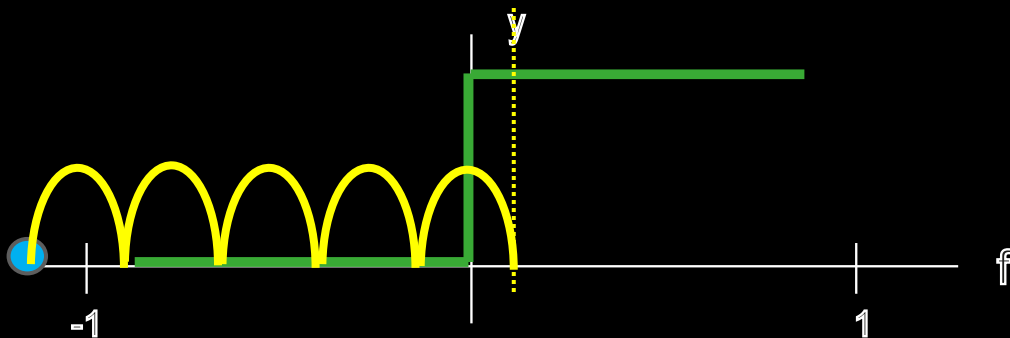
$$f = w_1 x_1 + w_2 x_2$$

$$w_1 = w_1 + \alpha$$

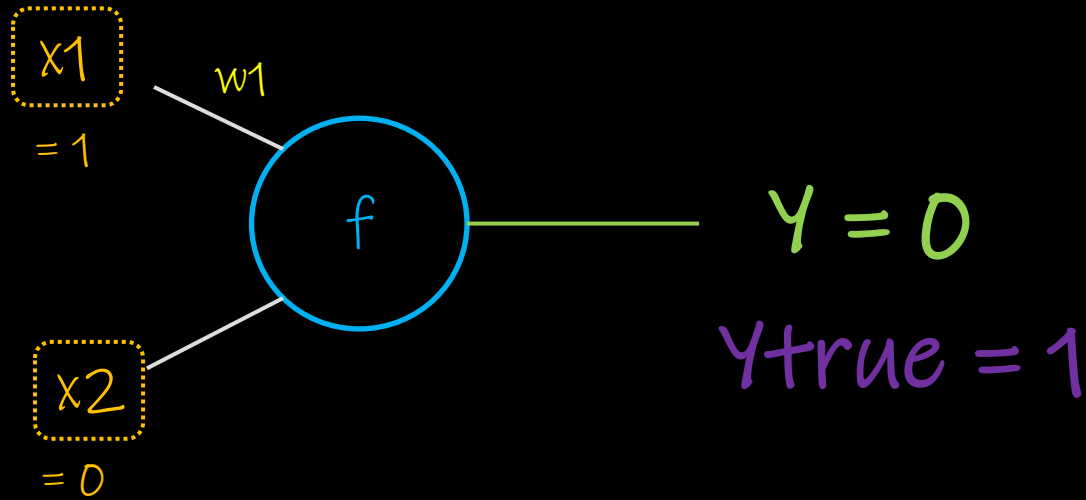
$$w_2 = w_2$$

$$(x_1 = 1)$$

$$(x_2 = 0)$$



The perceptron (1957)



$$W = W + \alpha (y_{true} - y) X$$

$$W = W + \alpha (1 - 1) X$$

$$W = W + 0$$

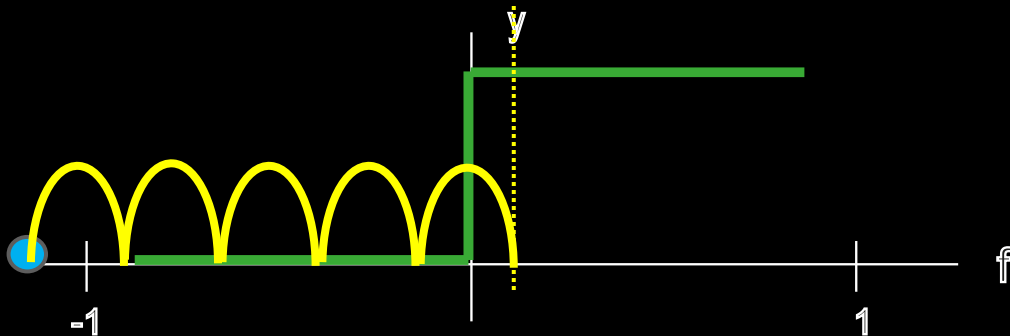
$$f = w_1 x_1 + w_2 x_2$$

$$w_1 = w_1 + \alpha$$

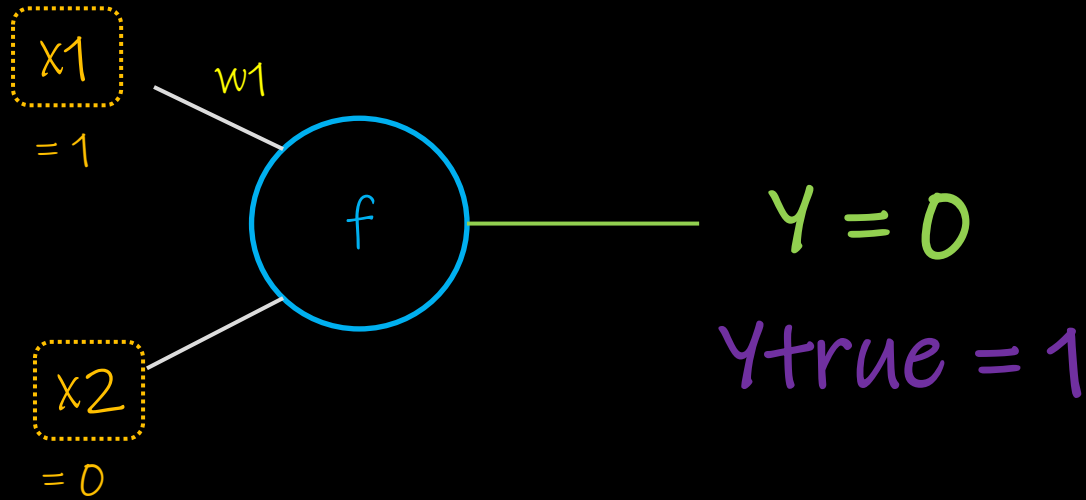
$$w_2 = w_2$$

$$(x_1 = 1)$$

$$(x_2 = 0)$$



The perceptron (1957)



$$W = W + a (y_{true} - y) X$$

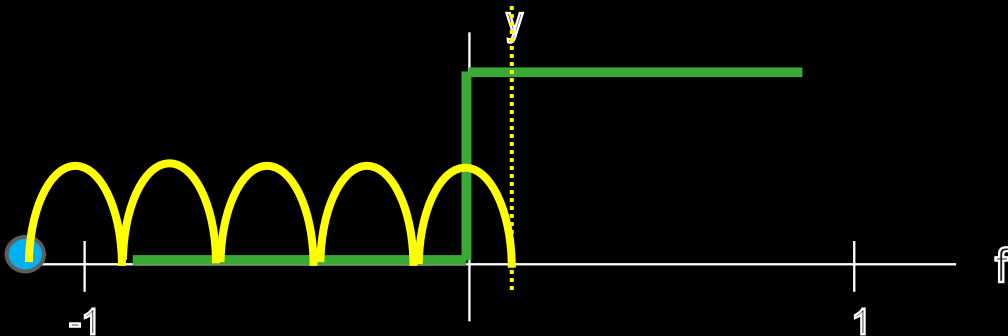
$$W = W + a (1 - 1) X$$

$$W = W + 0$$

$$f = w_1 x_1 + w_2 x_2$$

$$w_1 = w_1$$

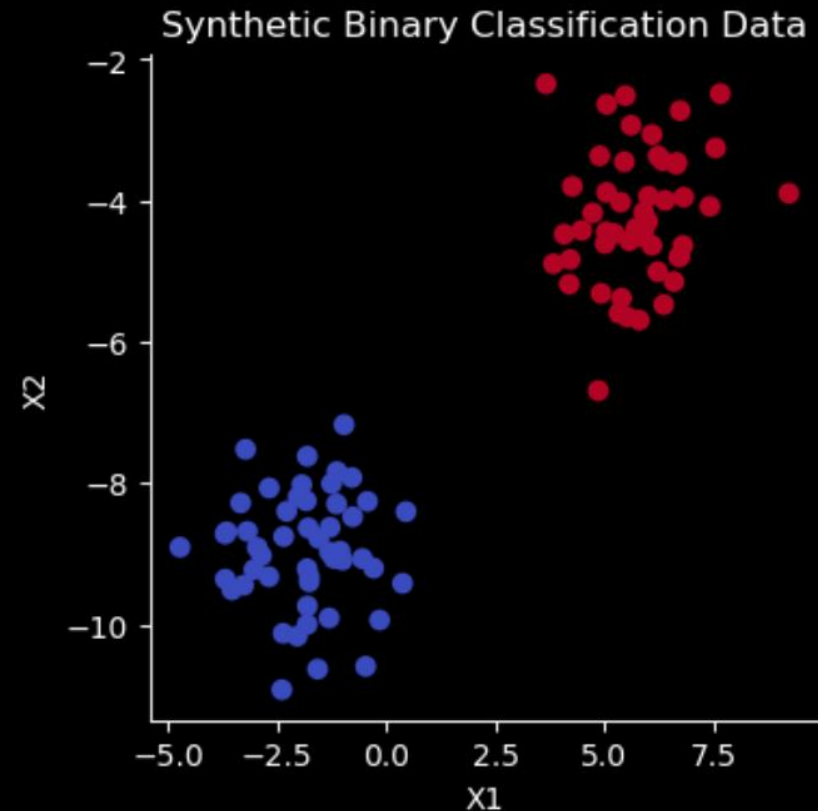
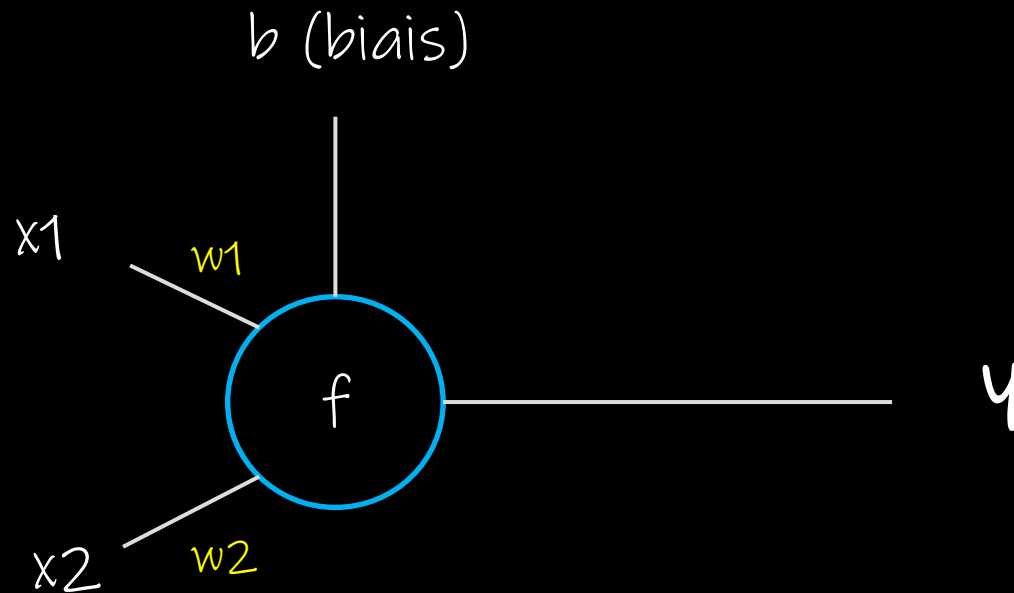
$$w_2 = w_2$$



The perceptron (1957) : example

The perceptron is the basic unit of neural networks.

It is a binary classification model capable of linearly separating two classes of points.

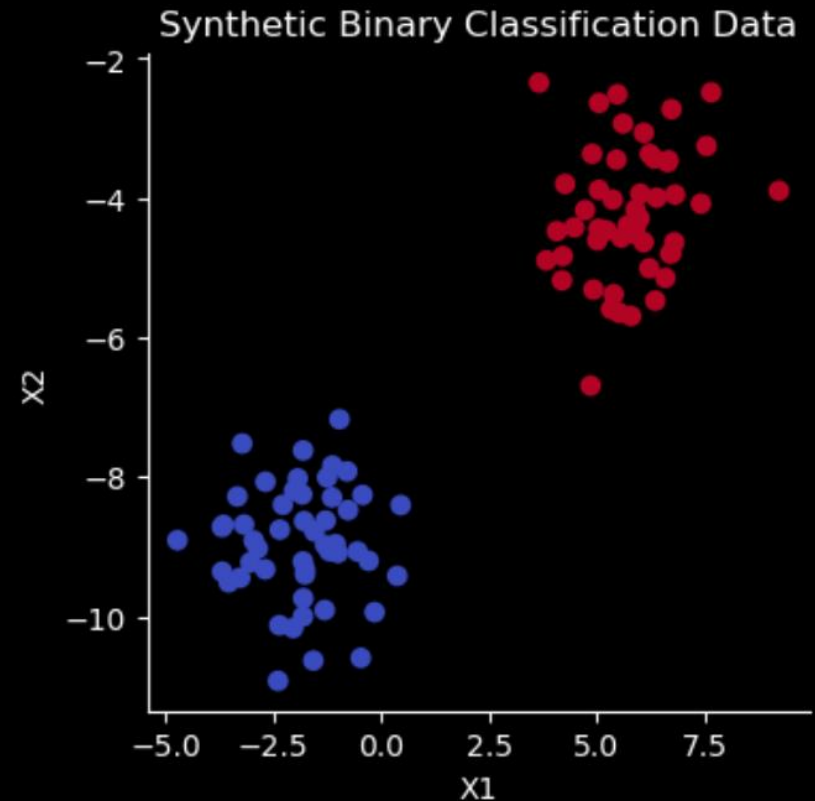


The perceptron (1957) : example

The perceptron is the basic unit of neural networks.

It is a binary classification model capable of linearly separating two classes of points.

y	X1	X2
1	7	-2
1	5	-4
0	-2,5	-10
0	0	-9
1	7,8	-3,8

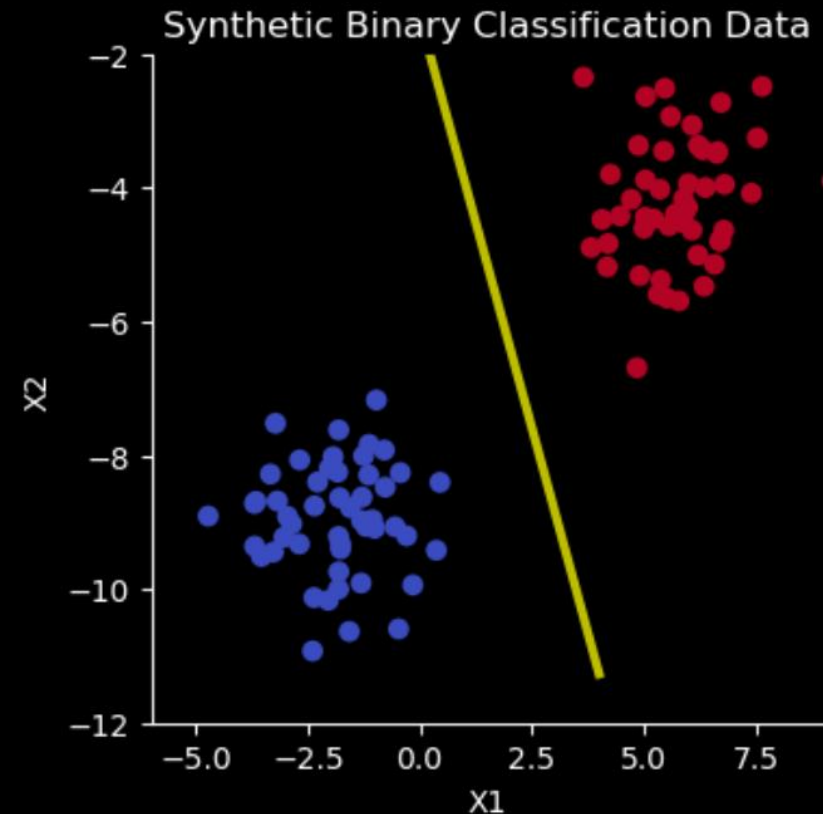


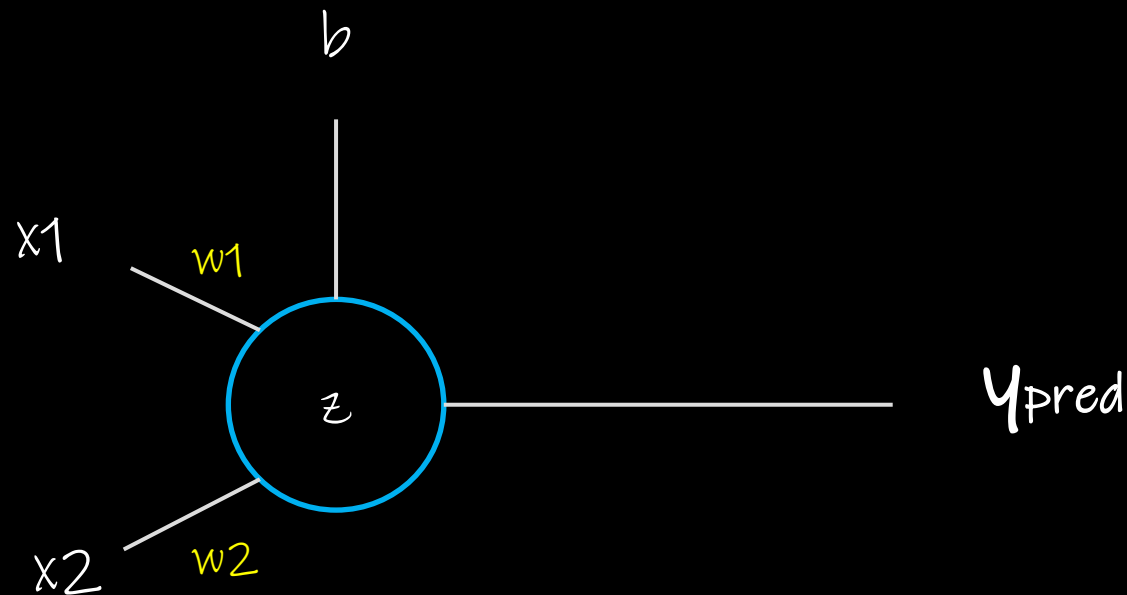
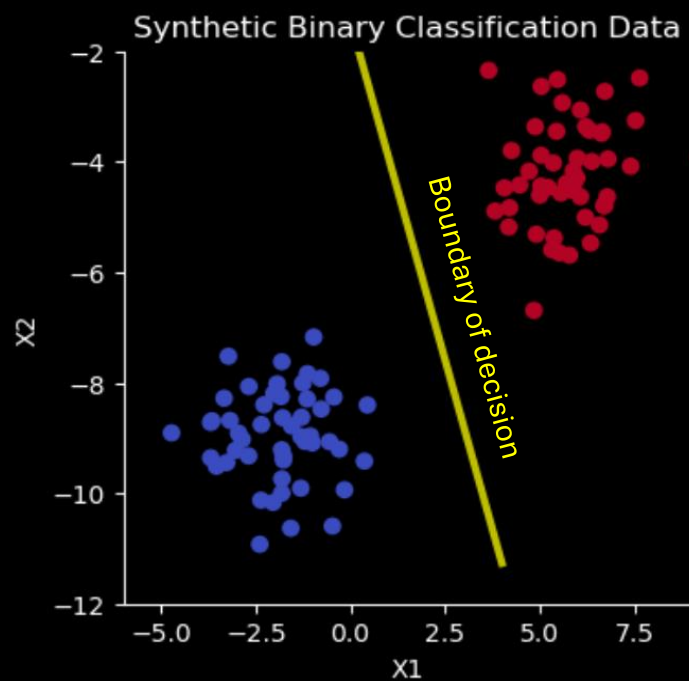
The perceptron (1957) : example

The perceptron is the basic unit of neural networks.

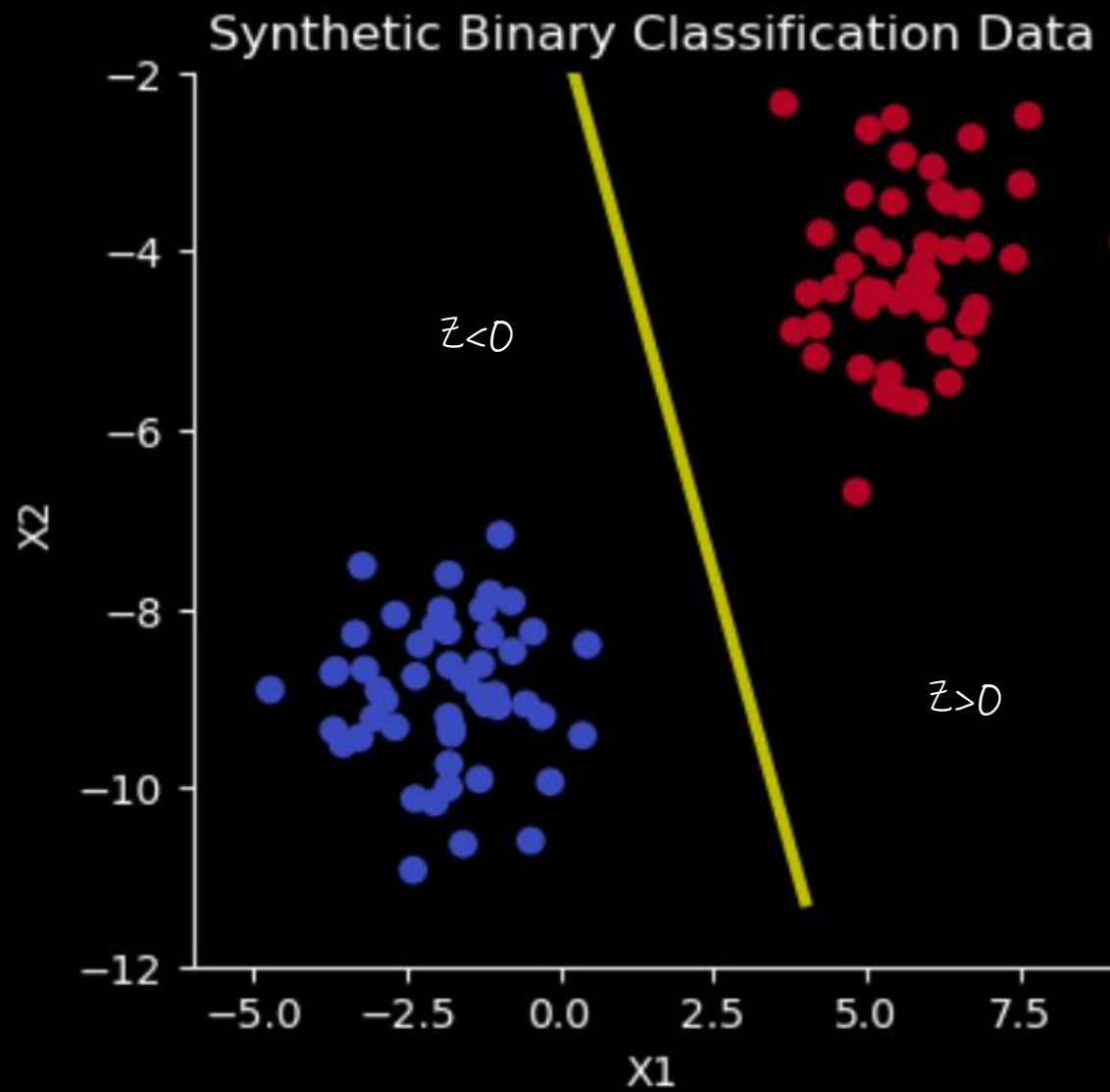
It is a binary classification model capable of linearly separating two classes of points.

y	X1	X2
1	7	-2
1	5	-4
0	-2,5	-10
0	0	-9
1	7,8	-3,8

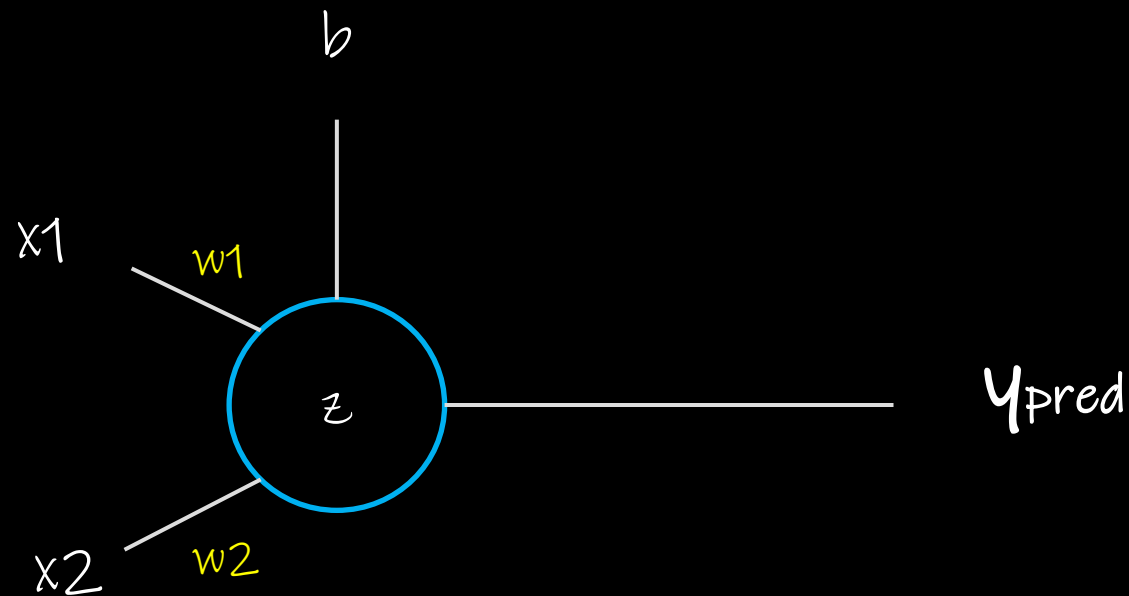
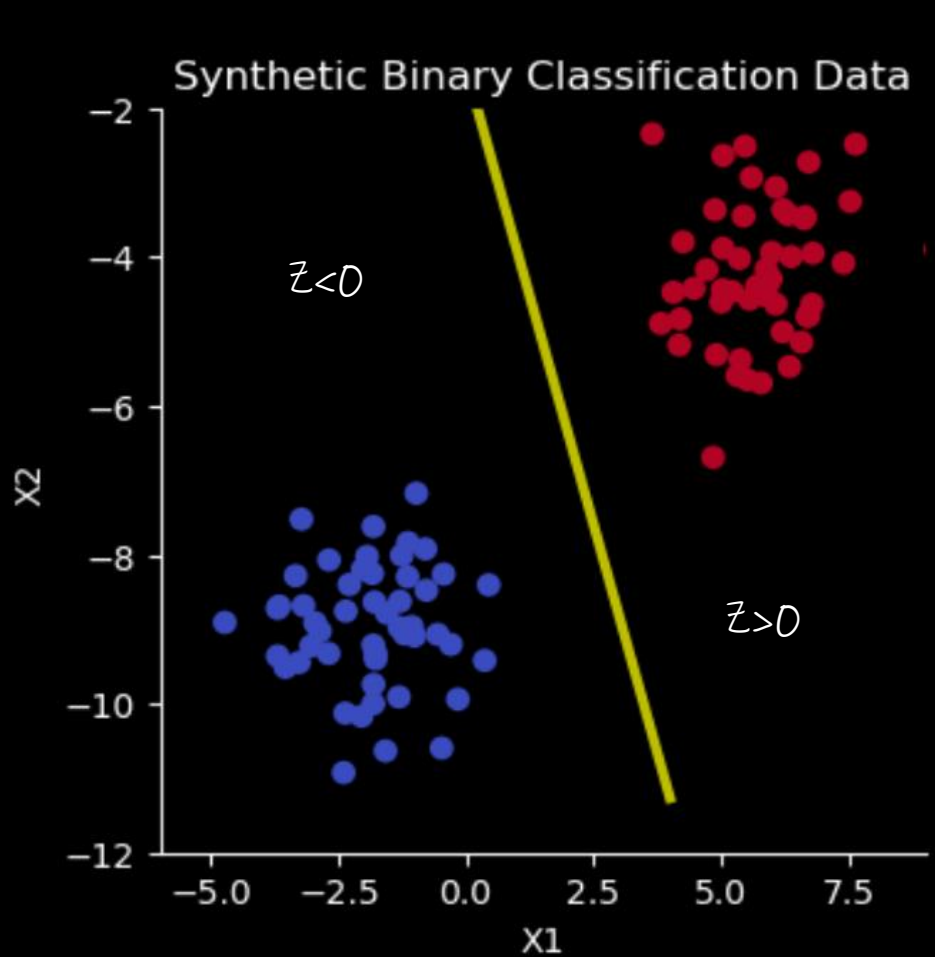




$$z(x_1, x_2) = w_1 x_1 + w_2 x_2 + b$$



$$z(x_1, x_2) = 0$$



$$z(x_1, x_2) = w_1 x_1 + w_2 x_2 + b$$

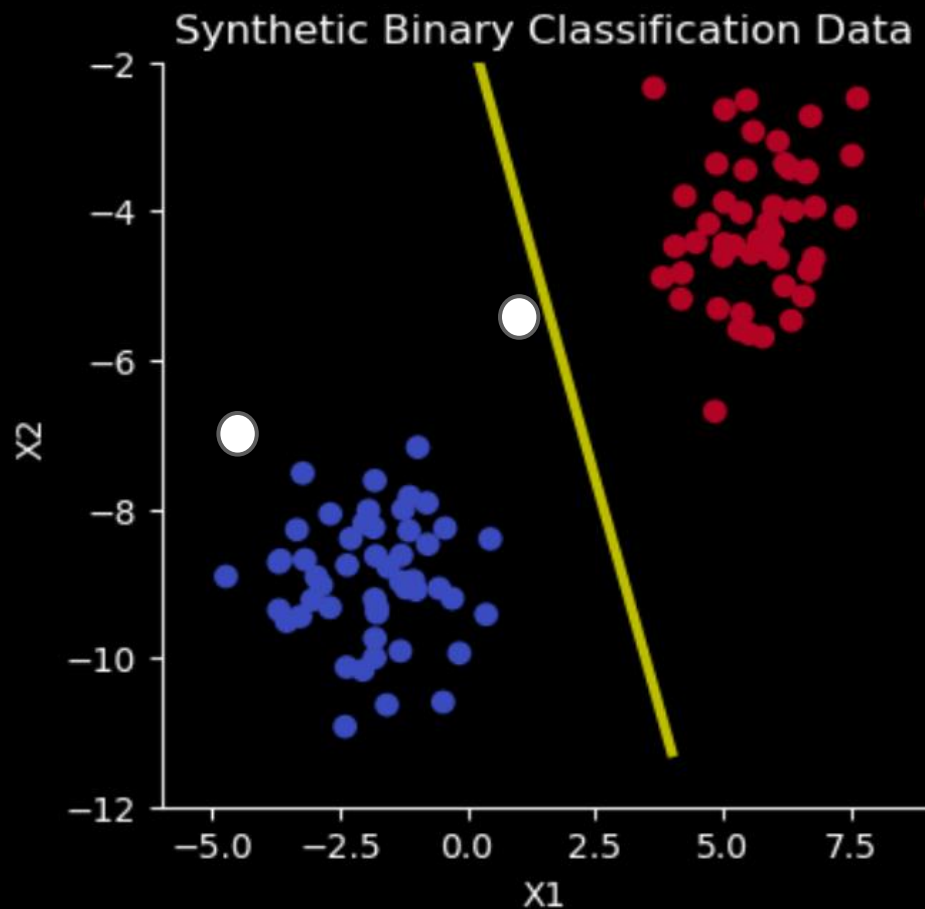
$$y_{pred} = 0 \text{ if } z < 0$$

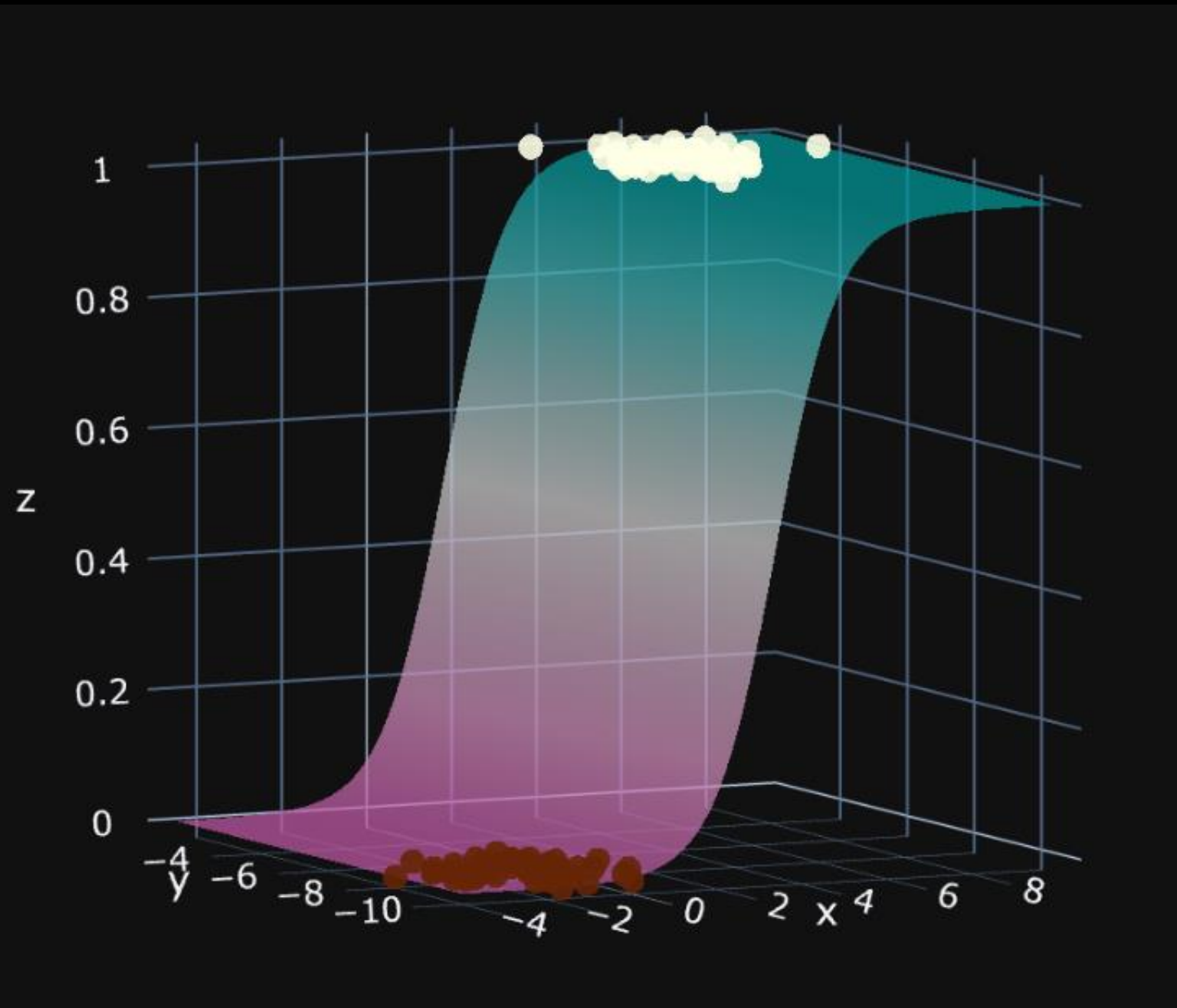
$$y_{pred} = 1 \text{ if } z > 0$$

To improve this model, it would be useful to associate each prediction with a probability. The farther a point is from the decision boundary, the more certain (or probable) it is that it belongs to its class. For that, we use an activation function that outputs a value approaching 0 or 1 as we move away from the decision boundary (where $z = 0$).

This function is the sigmoid function, also called the logistic function, defined as:

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

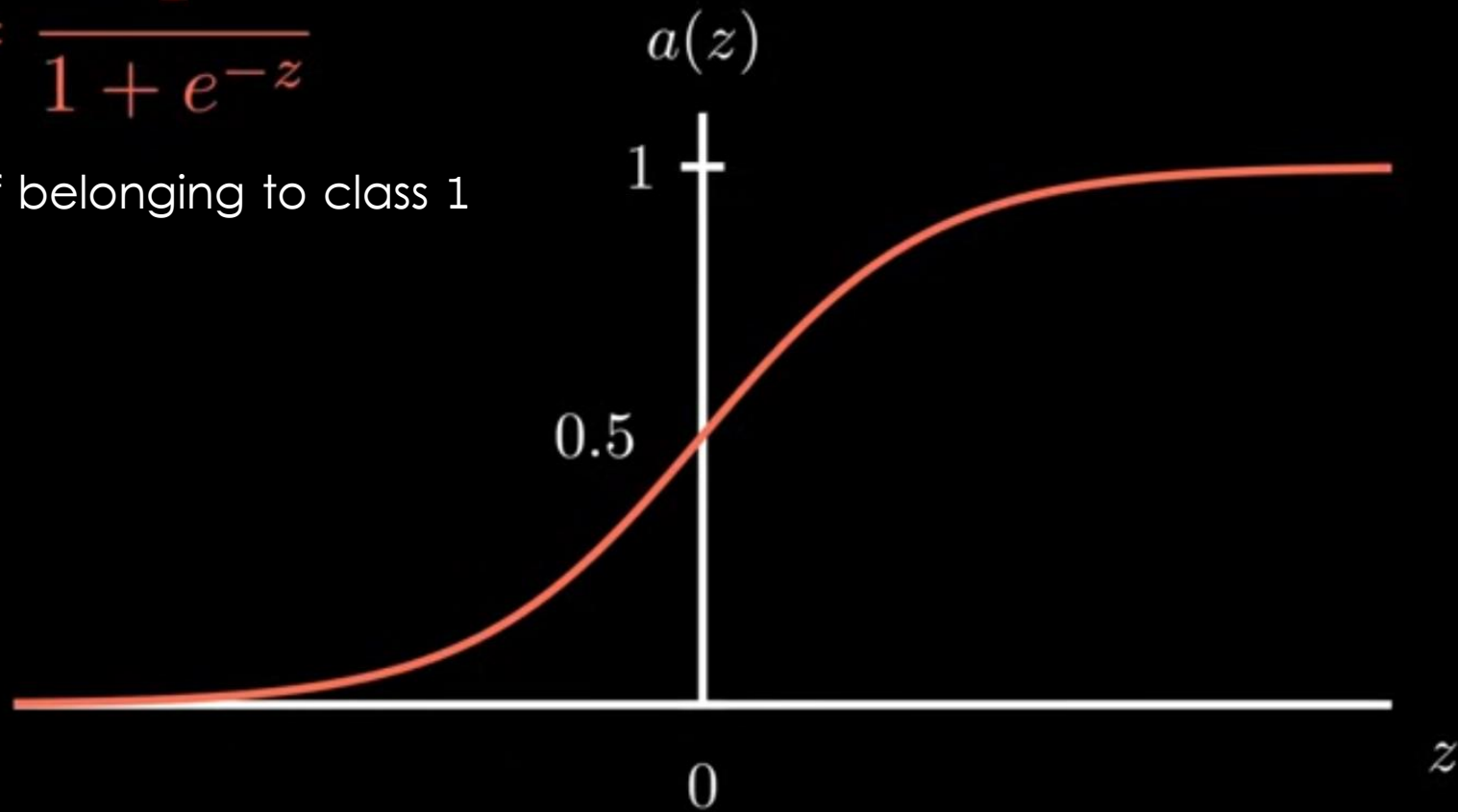




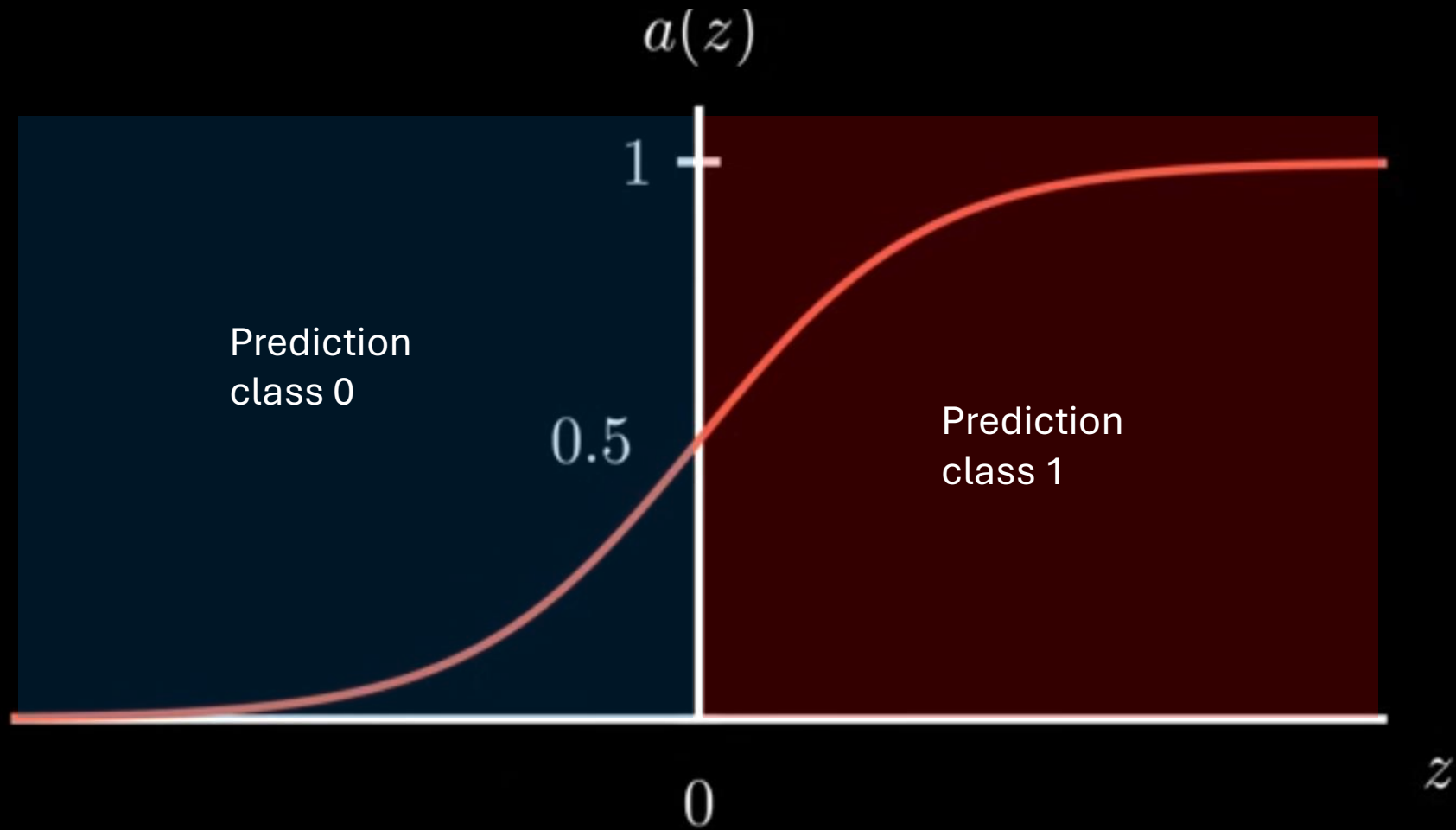
Sigmoide function (logistic)

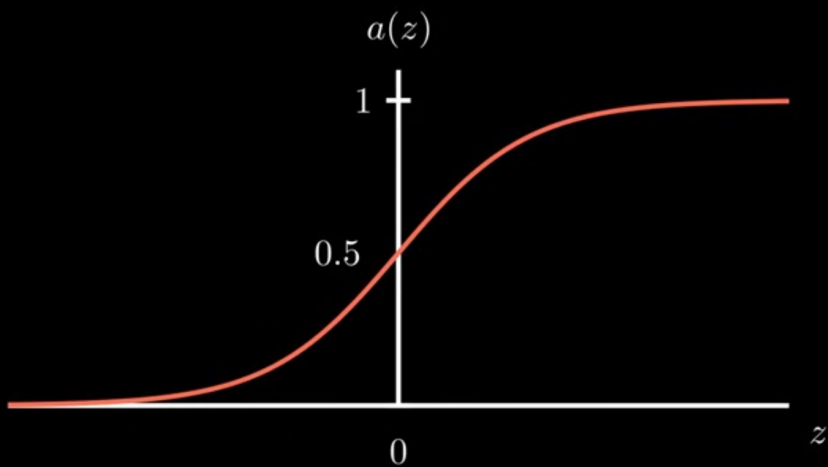
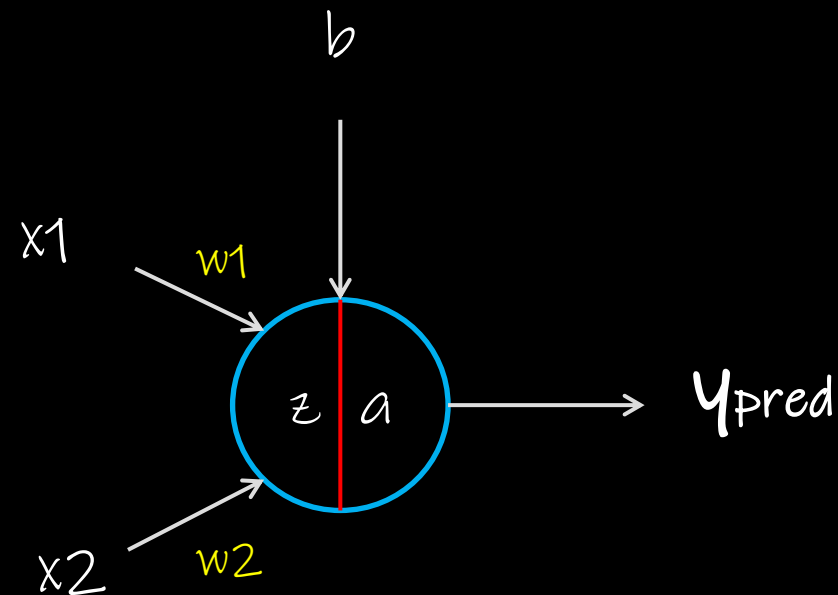
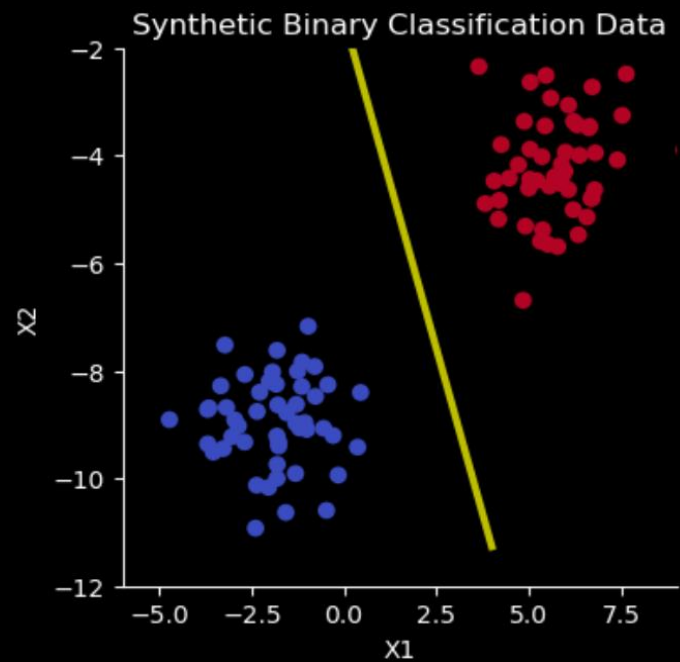
$$a(z) = \frac{1}{1 + e^{-z}}$$

a : probability of belonging to class 1



Sigmoide function (logistic)

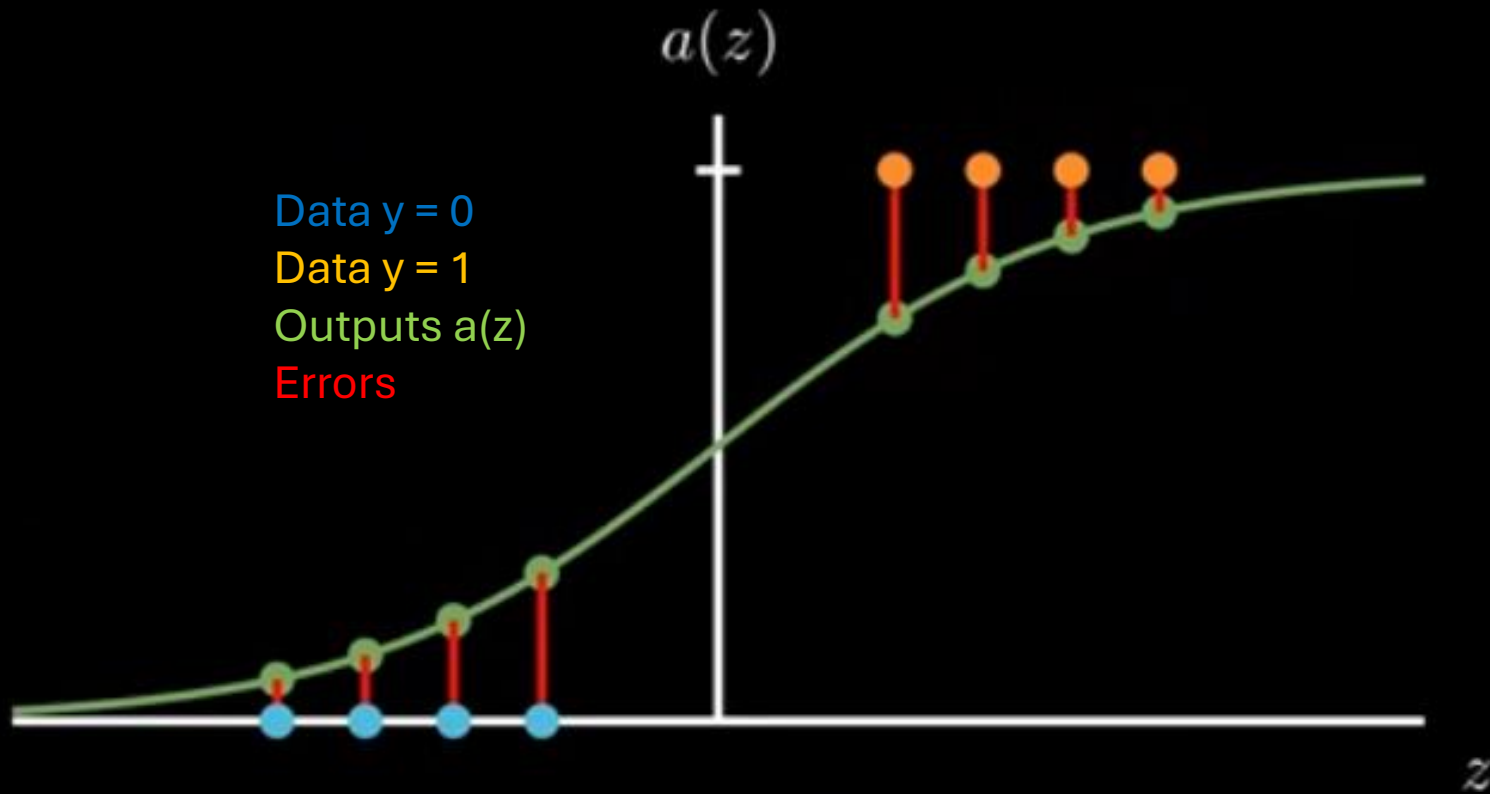




$$z = w_1 x_1 + w_2 x_2 + b$$

$$a = 1 / (1 + e^{-z})$$

The Loss Log Function



Log Loss

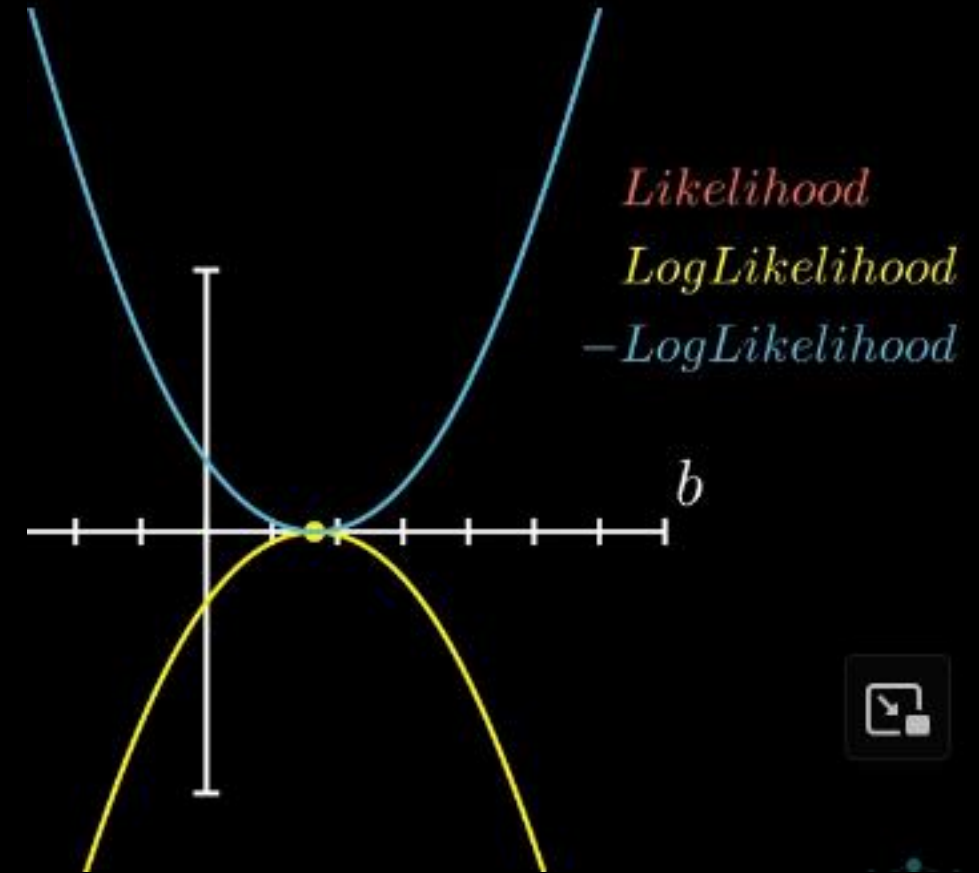
$$L = -\frac{1}{m} \sum_{i=1}^m y_i \log(a_i) + (1 - y_i) \log(1 - a_i)$$

The Loss Log Function

Logarithmic Loss, commonly known as Log Loss or Cross-Entropy Loss, is a crucial metric in machine learning, particularly in classification problems. It quantifies the performance of a classification model by measuring the difference between predicted probabilities and actual outcomes. In this article, we will delve into the concept of Log Loss, its significance, and provide a practical example using Python and scikit-learn.

What is Log Loss?

Log Loss is a logarithmic transformation of the likelihood function, primarily used to evaluate the performance of probabilistic classifiers. Unlike other metrics such as accuracy, Log Loss takes into account the uncertainty of predictions by penalizing models more heavily for confidently incorrect predictions.



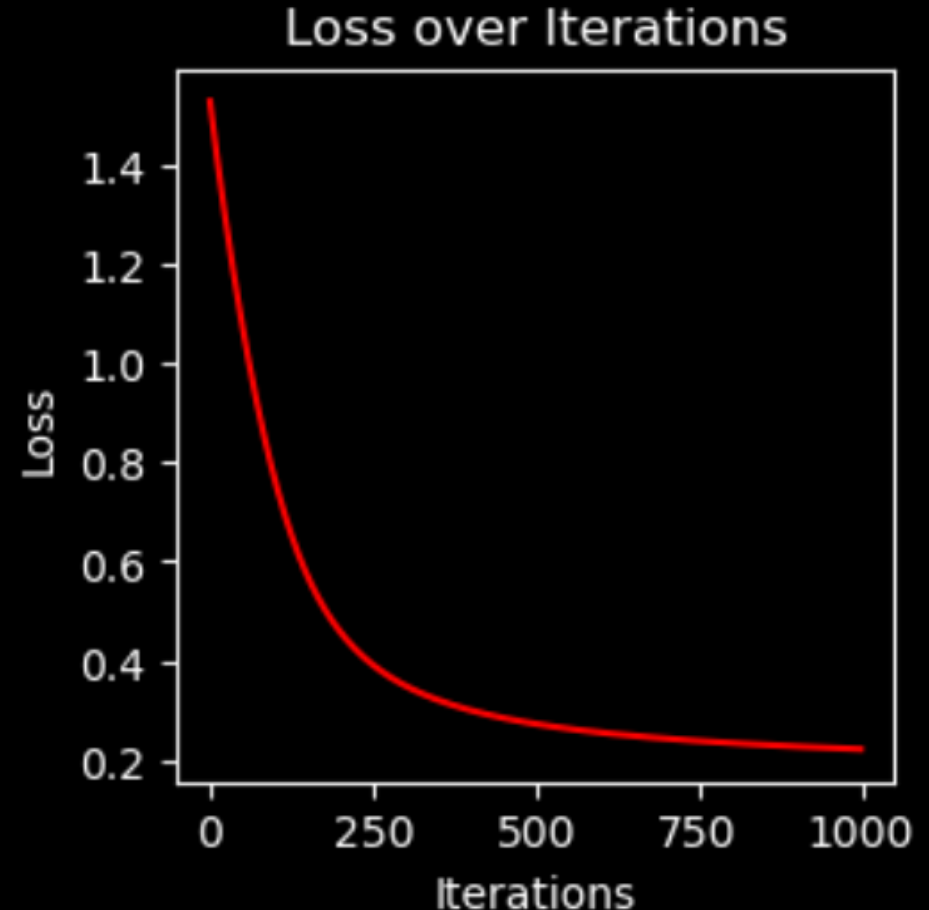
$$\text{Maximise } f(x) = \text{minimise } -f(x)$$

The Loss Log Function

Logarithmic Loss, commonly known as Log Loss or Cross-Entropy Loss, is a crucial metric in machine learning, particularly in classification problems. It quantifies the performance of a classification model by measuring the difference between predicted probabilities and actual outcomes. In this article, we will delve into the concept of Log Loss, its significance, and provide a practical example using Python and scikit-learn.

What is Log Loss?

Log Loss is a logarithmic transformation of the likelihood function, primarily used to evaluate the performance of probabilistic classifiers. Unlike other metrics such as accuracy, Log Loss takes into account the uncertainty of predictions by penalizing models more heavily for confidently incorrect predictions.

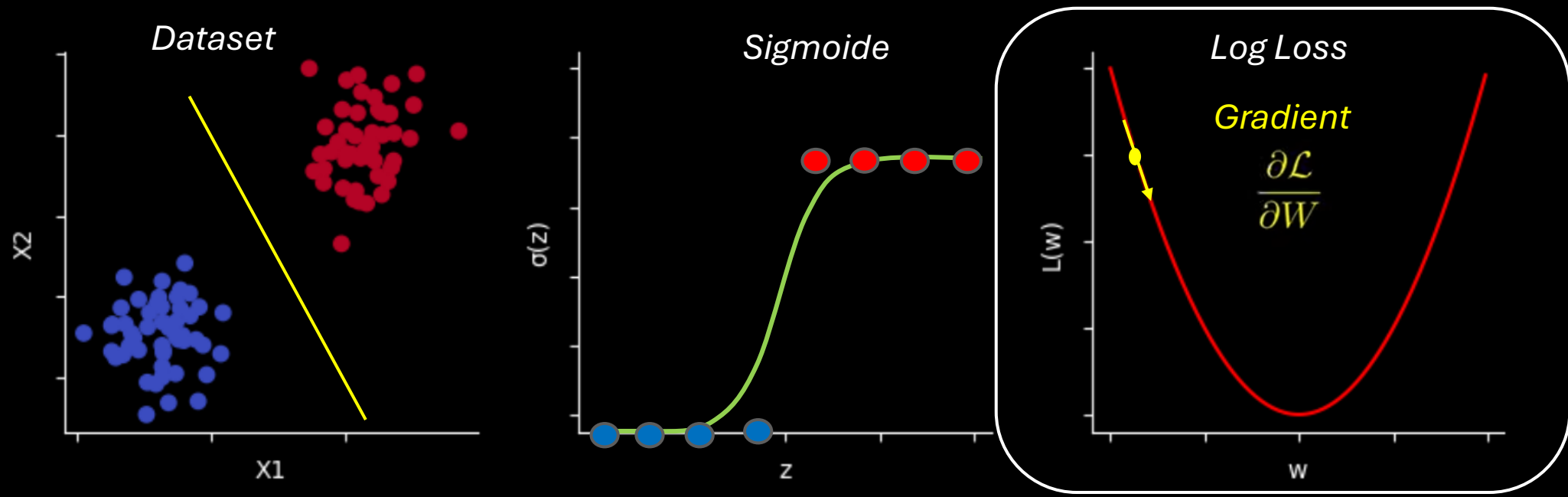


The gradient descent !!!

Gradient descent is one of the most widely used optimization algorithms in machine learning and deep learning.

It adjusts the parameters w and b in order to minimize the cost.

Repeating this process iteratively allows us to reach the minimum of the cost function. And since log-loss is convex, there's only one global minimum, so gradient descent will work well.



The gradient descent !!!

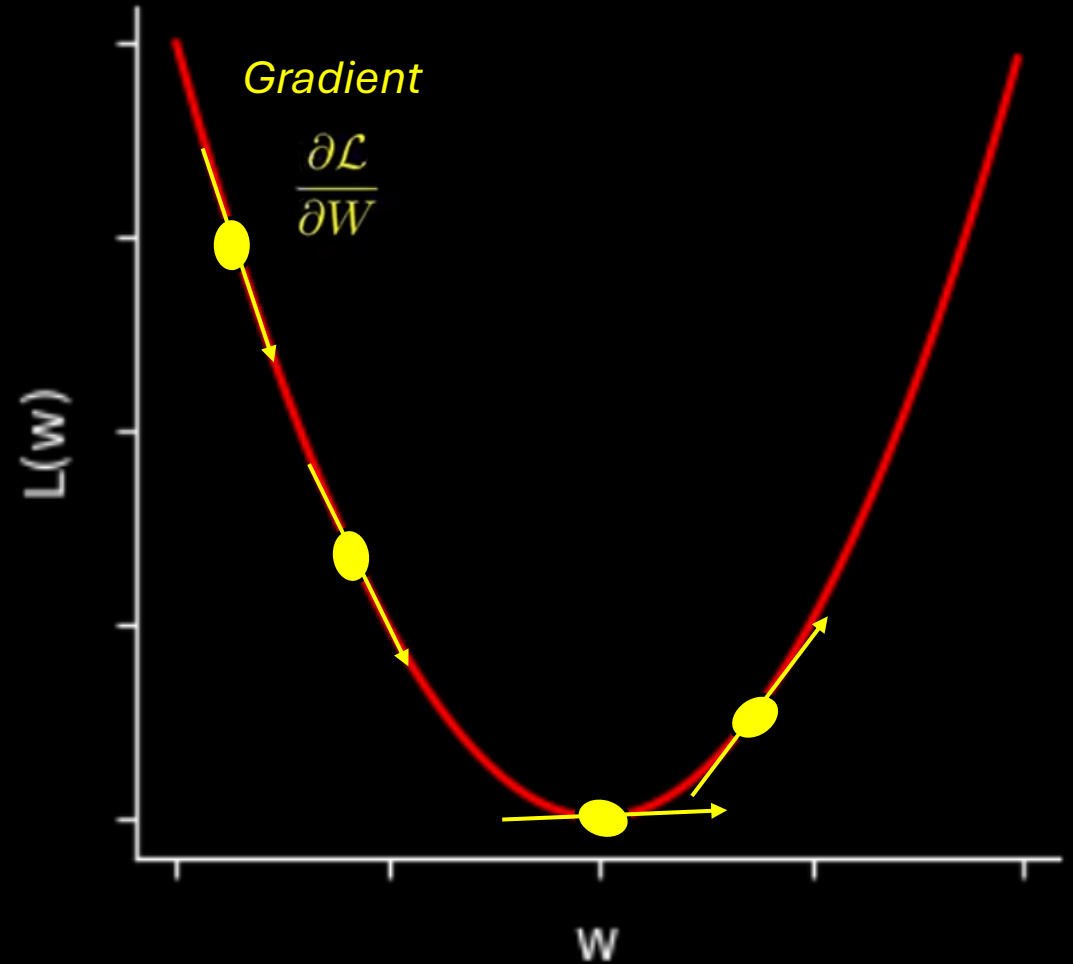
The idea is simple:

- Compute the gradient (the slope) of the cost with respect to each parameter.
- Update the parameters in the opposite direction of the slope.

$$W_{t+1} = W_t - \alpha \cdot \frac{\partial L}{\partial W_t}$$

where α is the learning rate, a small positive number.

Log Loss



Repeating this step moves us closer and closer to the minimum of the cost function.

And since log-loss is convex, there's only one global minimum — no risk of getting stuck in local traps.

The gradient descent !!!

$$W_{t+1} = W_t - \alpha \cdot \frac{\partial L}{\partial W_t}$$

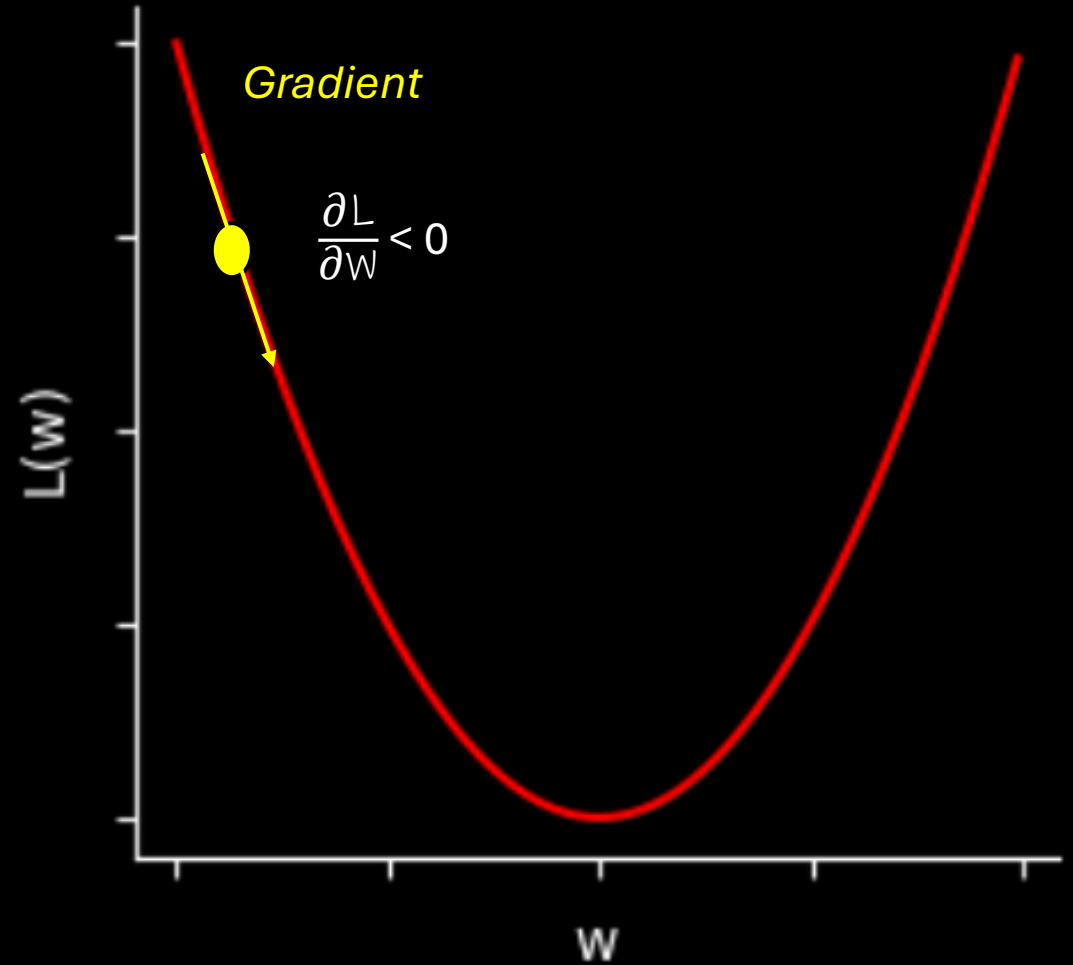
$$W_{t+1} = W_t - \boxed{\alpha \cdot \frac{\partial L}{\partial W_t}}$$

< 0

$$W_{t+1} = W_t + \text{something positive}$$

$$\Rightarrow W_{t+1} > W_t$$

Log Loss



Repeating this step moves us closer and closer to the minimum of the cost function.
And since log-loss is convex, there's only one global minimum — no risk of getting stuck in local traps.

The gradient descent !!!

$$W_{t+1} = W_t - \alpha \cdot \frac{\partial L}{\partial W_t}$$

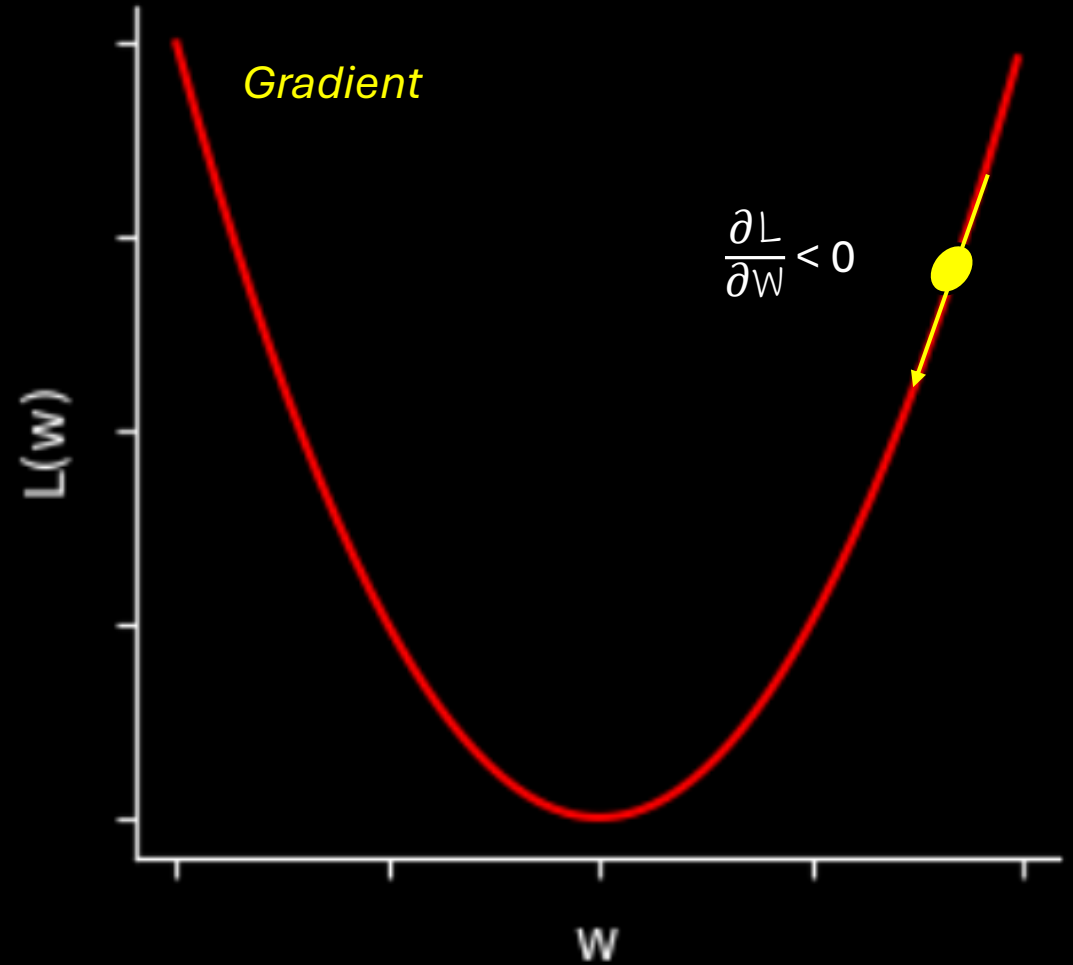
$$W_{t+1} = W_t - \boxed{\alpha \cdot \frac{\partial L}{\partial W_t}}$$

> 0

$$W_{t+1} = W_t + \text{something negative}$$

$$\Rightarrow W_{t+1} < W_t$$

Log Loss

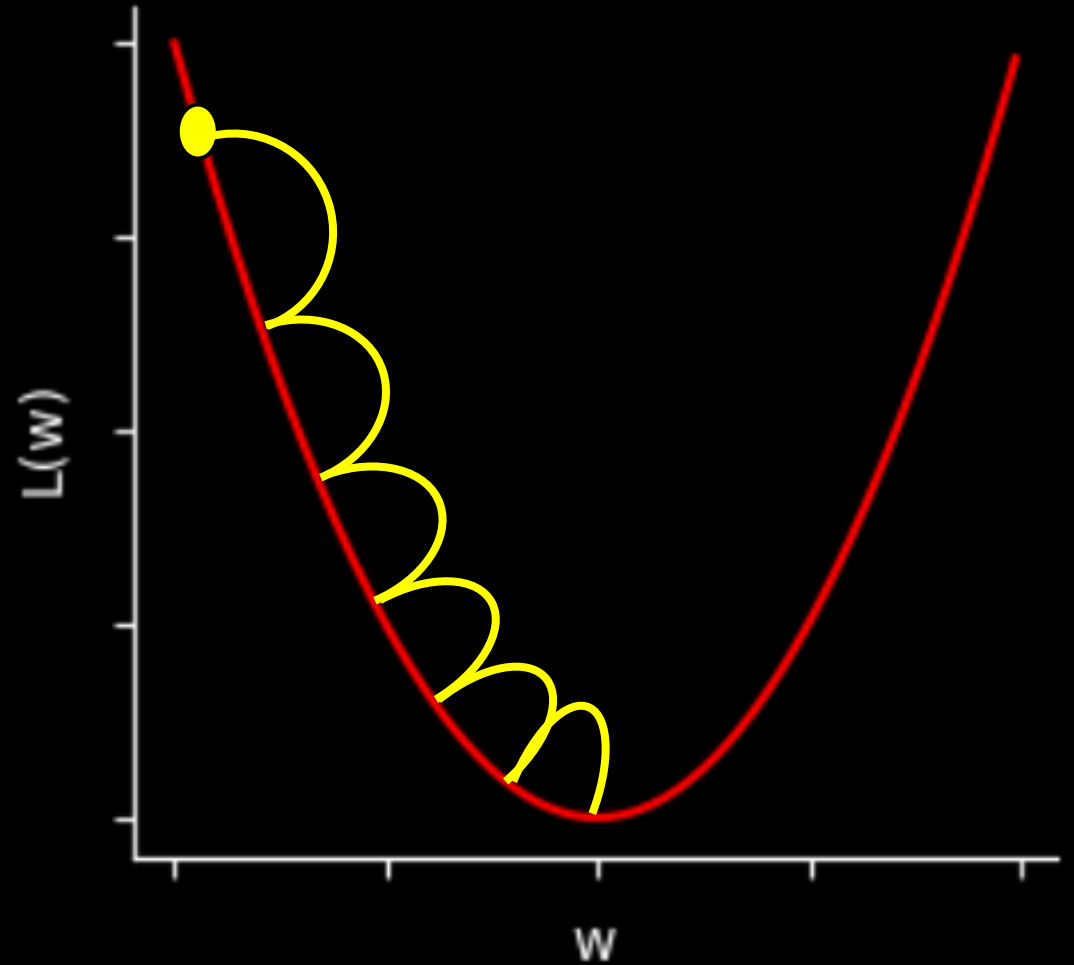


Repeating this step moves us closer and closer to the minimum of the cost function.
And since log-loss is convex, there's only one global minimum — no risk of getting stuck in local traps.

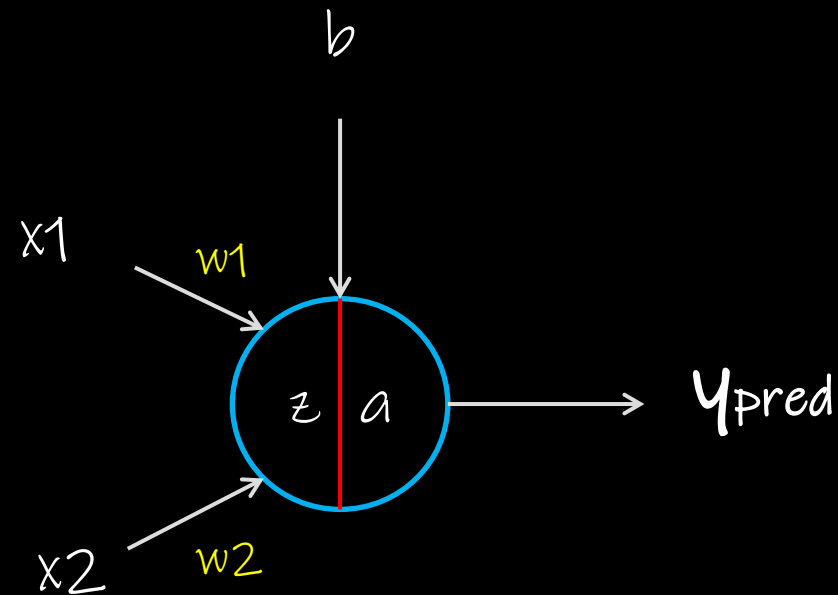
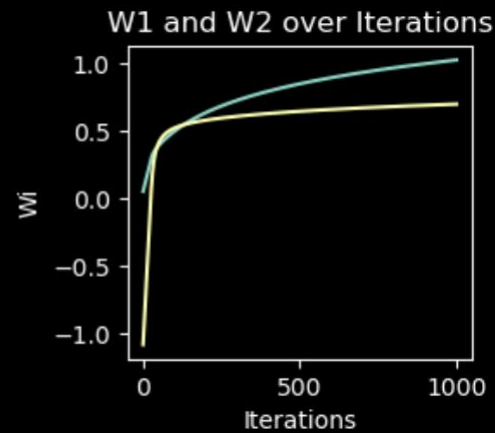
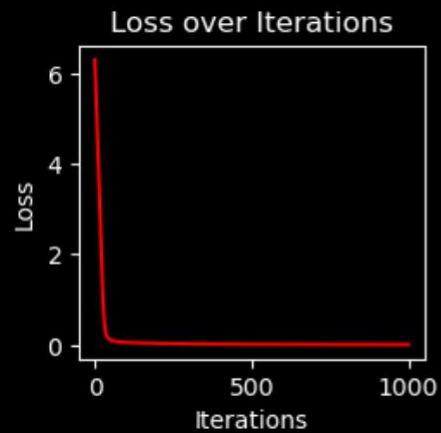
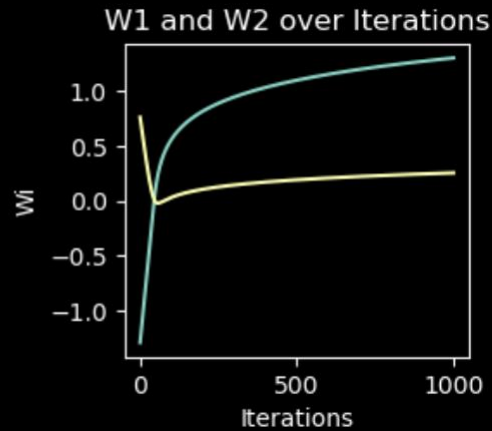
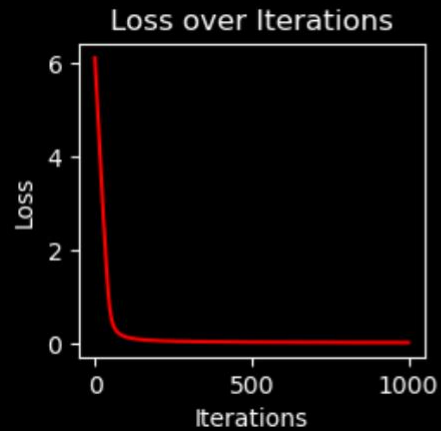
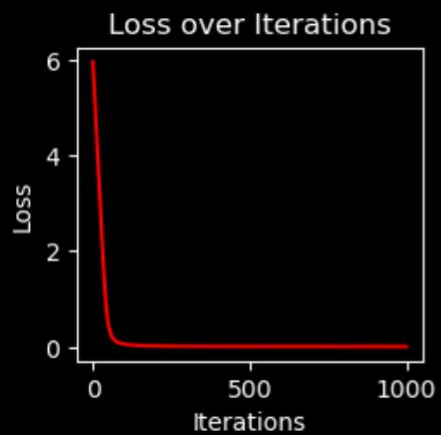
The gradient descent !!!

$$w_{t+1} = w_t - \alpha \cdot \frac{\partial L}{\partial w}$$

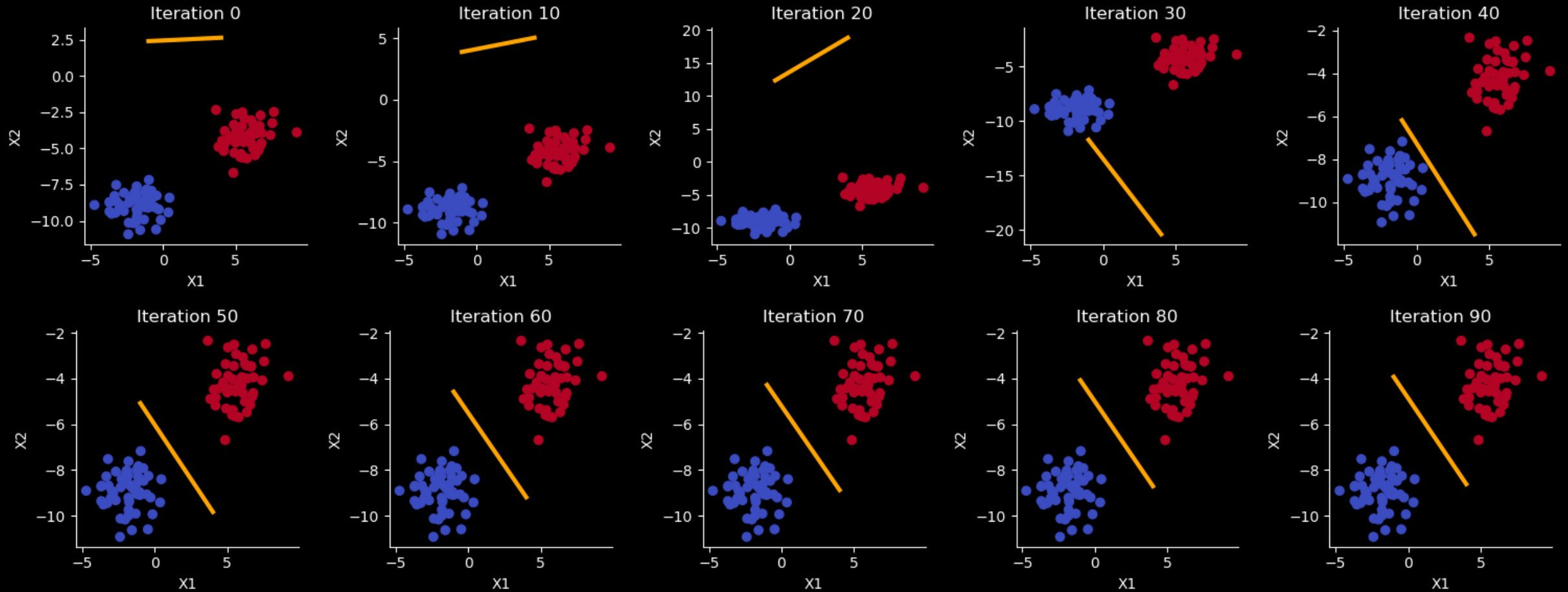
Log Loss



Repeating this step moves us closer and closer to the minimum of the cost function.
And since log-loss is convex, there's only one global minimum — no risk of getting stuck in local traps.



Evolution of the boundary of decision as a function of iterations and w



1950 : Alan Turing

Computing machinery and
Intelligence

1949 : Donald Hebb

'The Organization of Behavior',
introducing the concept that
would become Hebbian
learning: "Neurons that fire
together, wire together."



Donald Hebb

1957 : F. Rosenblatt

Invention of the perceptron



Frank Rosenblatt

1957 : F. Rosenblatt
Invention of the perceptron

1969 : XOR Problem
Minsky & Papert



Frank Rosenblatt

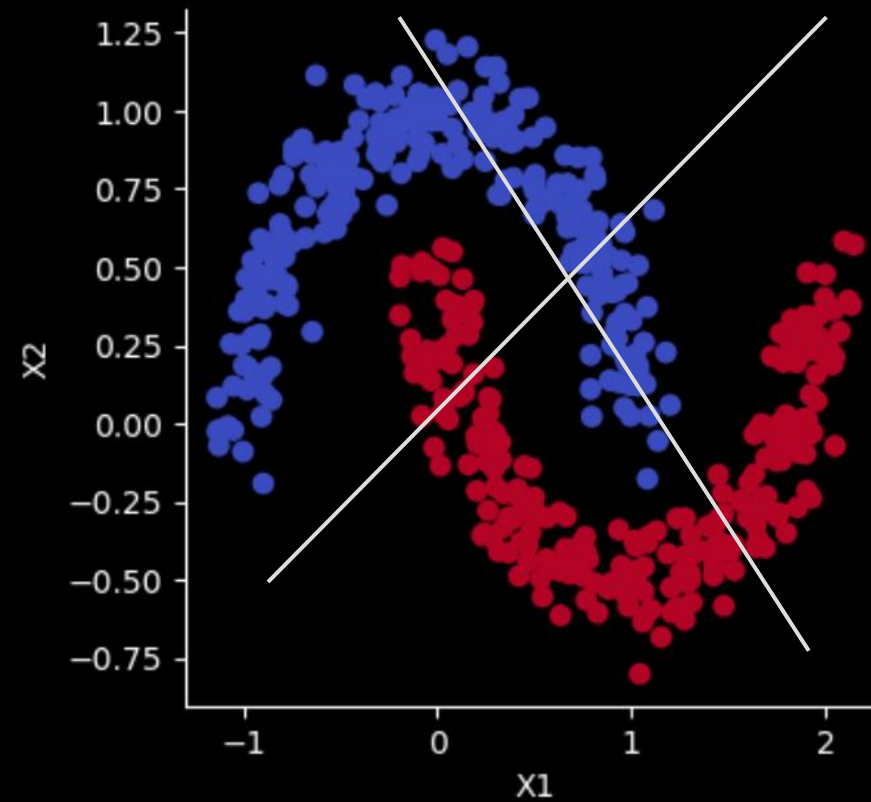
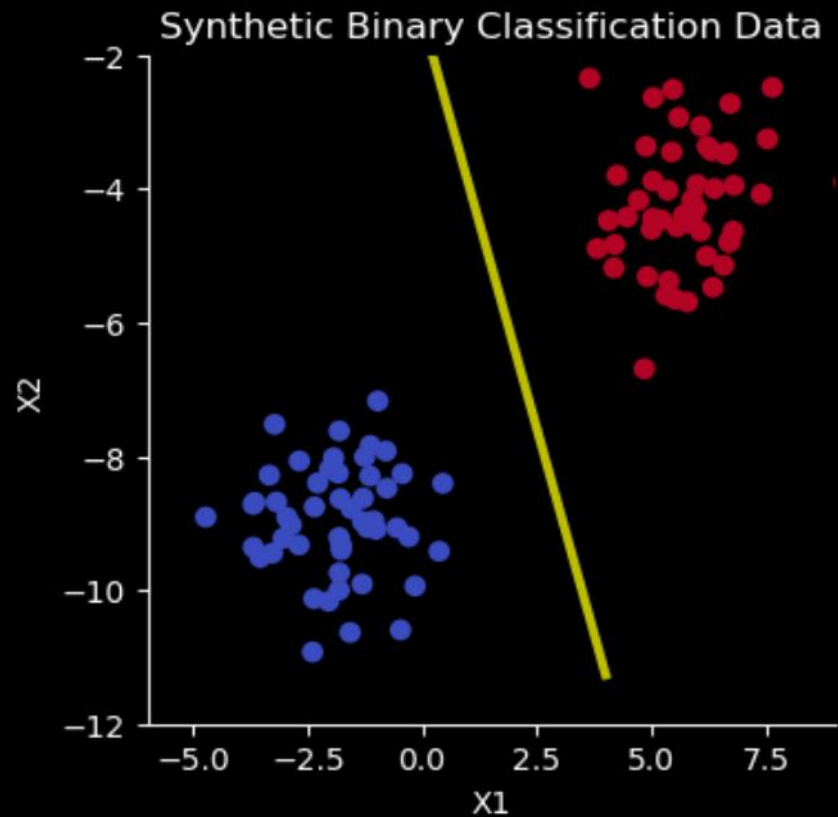
The XOR, or “exclusive or”, problem is a classic problem in ANN research. It is the problem of using a neural network to predict the outputs of XOR logic gates given two binary inputs. An XOR function should return a true value if the two inputs are not equal and a false value if they are equal. All possible inputs and predicted outputs are shown in figure 1.

Input 1	Input 2	Output
0	0	0
0	1	1
1	1	0
1	0	1

XOR is a classification problem and one for which the expected outputs are known in advance. It is therefore appropriate to use a supervised learning approach.

On the surface, XOR appears to be a very simple problem, however, Minsky and Papert (1969) showed that this was a big problem for neural network architectures of the 1960s, known as perceptrons.

The perceptron is not working for non linear data



1957 : F. Rosenblatt
Invention of the perceptron



Frank Rosenblatt

1969 : XOR Problem
Minsky & Papert

1974

First winter of AI

1980

Minsky, M. Papert, S. (1969). Perceptron: an introduction to computational geometry. The MIT Press, Cambridge, expanded edition, 19(88), 2.

First winter of AI

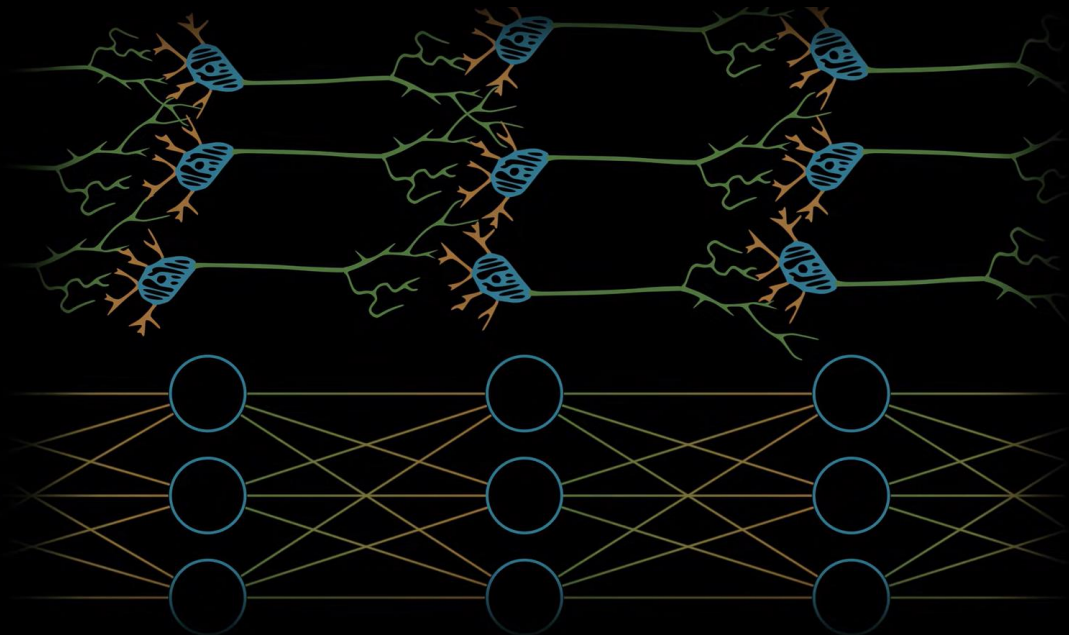
1980

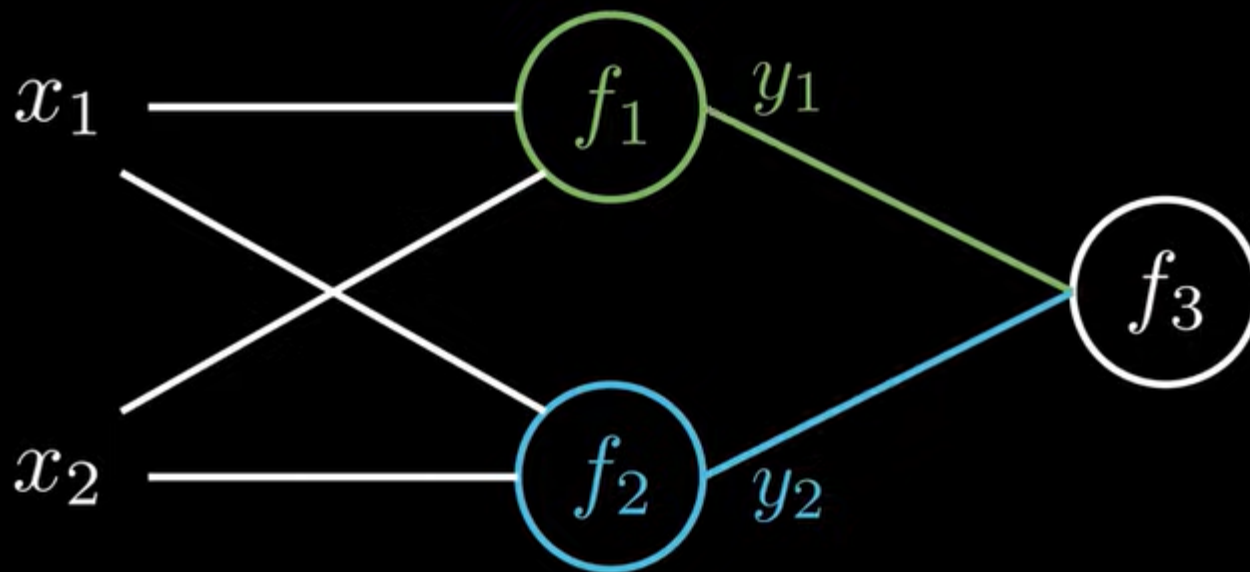
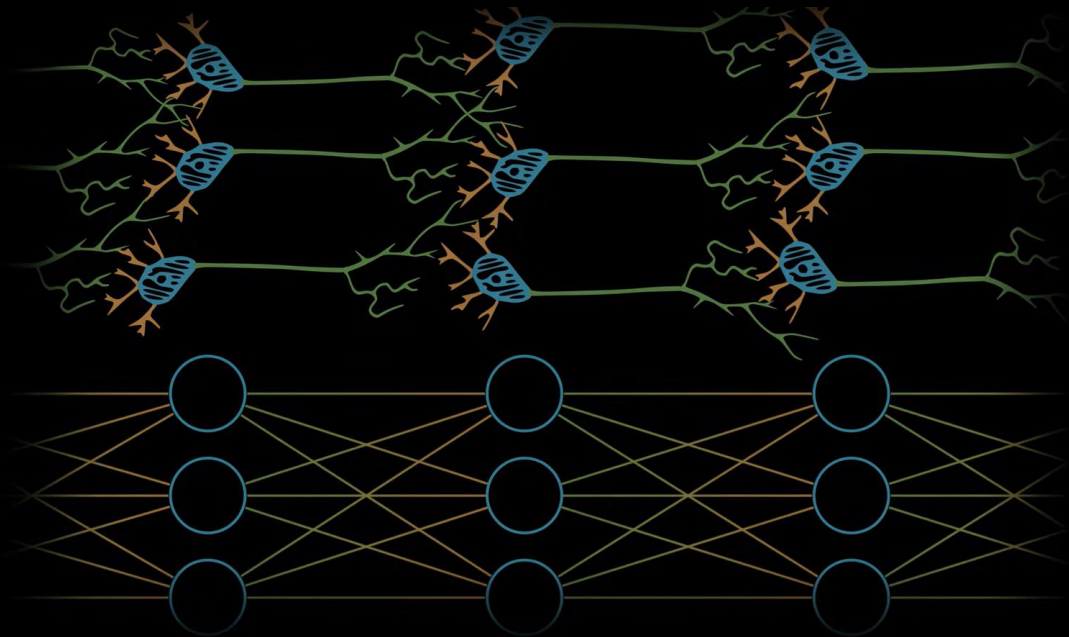
1986 : The Multilayer
Perceptron (MLP)



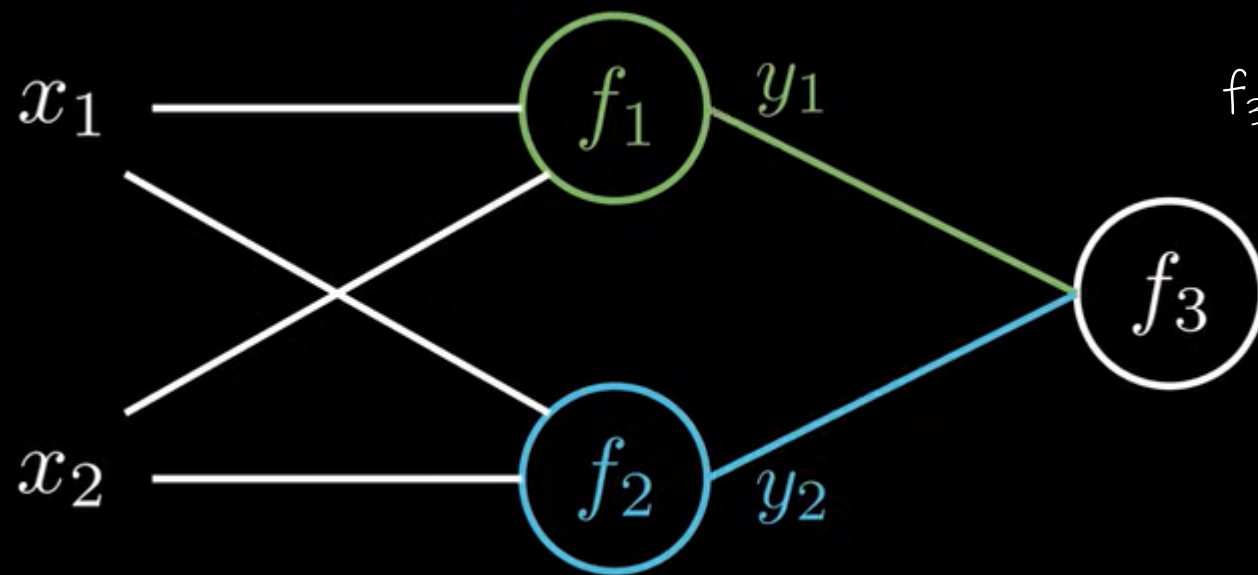
Geoffrey Hinton

The real first ANN





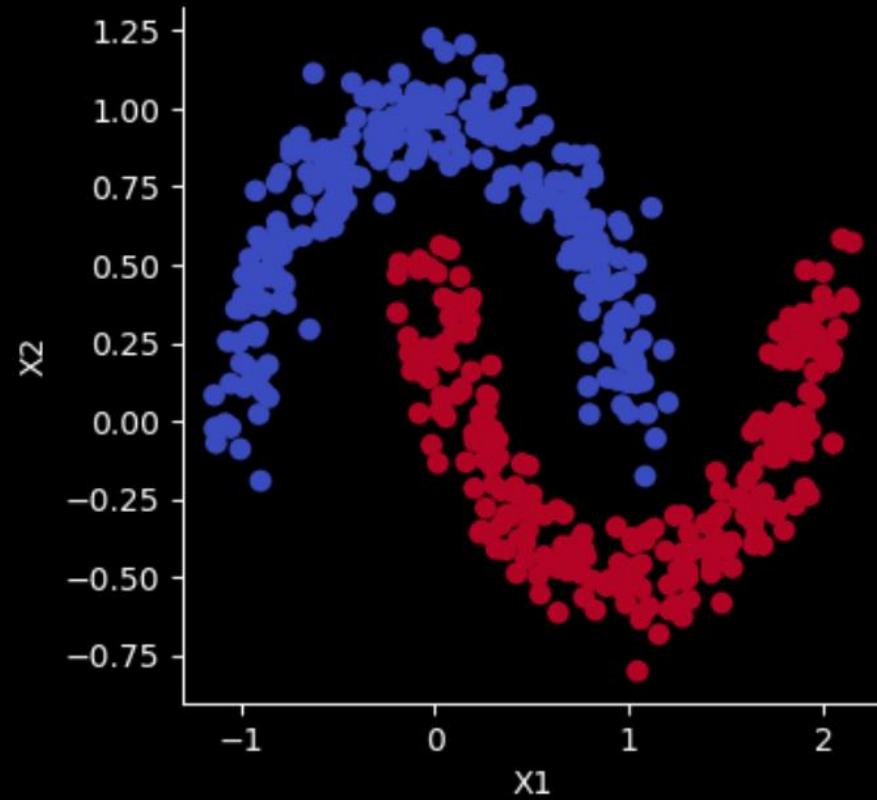
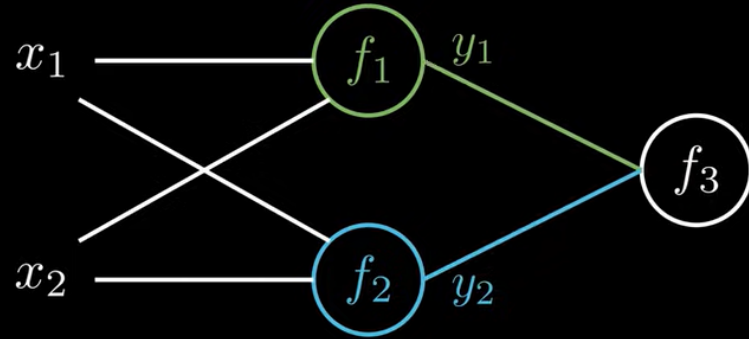
$$f_1 = w_{11}x_1 + w_{12}x_2 + 1 \quad \begin{cases} y_1 = 1 & \text{if } f_1 > 0 \\ y_1 = 0 & \text{else} \end{cases}$$



$$f_3 = w_{31}x_1 + w_{32}x_2 + b_3 \quad \begin{cases} y_3 = 1 & \text{if } f_3 > 0 \\ y_3 = 0 & \text{else} \end{cases}$$

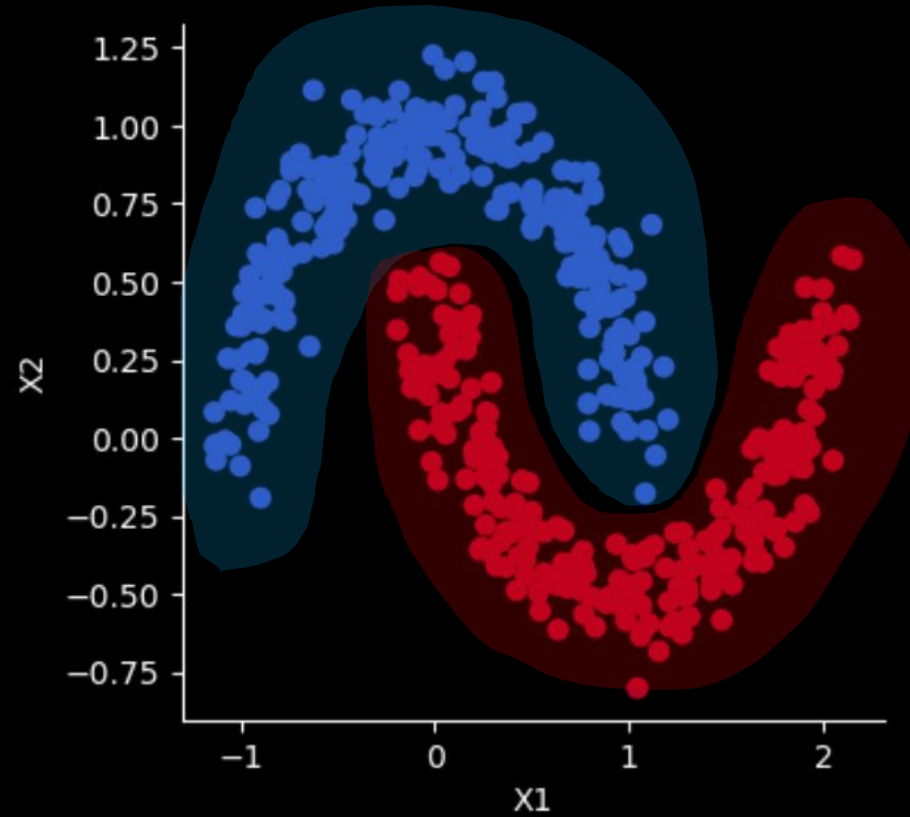
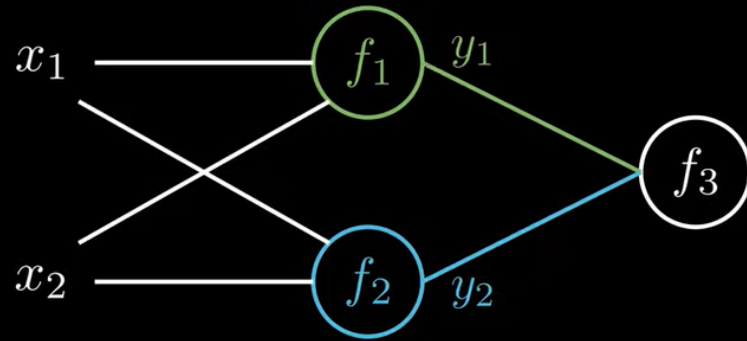
$$f_2 = w_{21}x_1 + w_{22}x_2 + b_2 \quad \begin{cases} y_2 = 1 & \text{if } f_2 > 0 \\ y_2 = 0 & \text{else} \end{cases}$$

The Multilayer perceptron (1986)

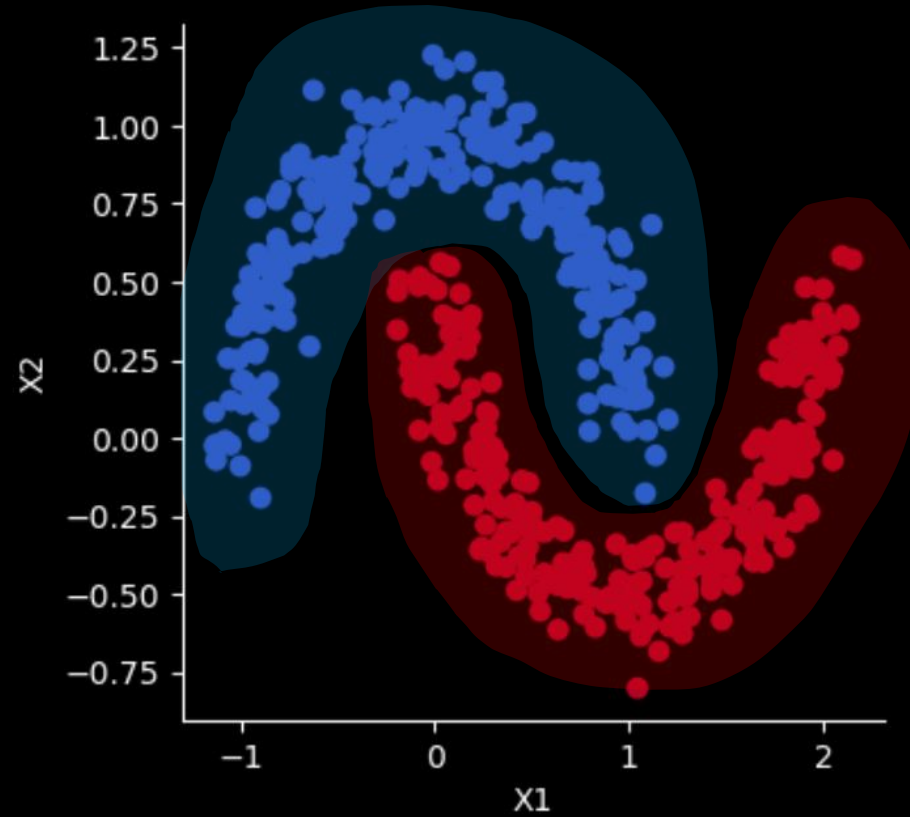
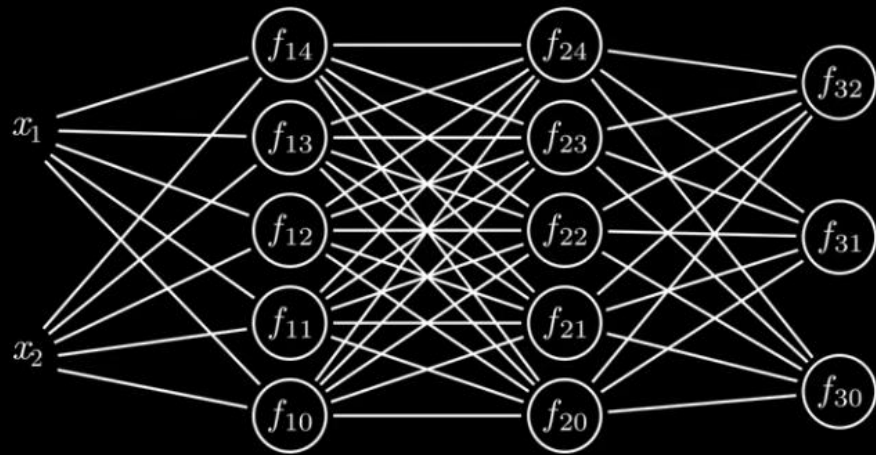


The Multilayer perceptron (1986)

First ANN 3 neurons on 2 layers

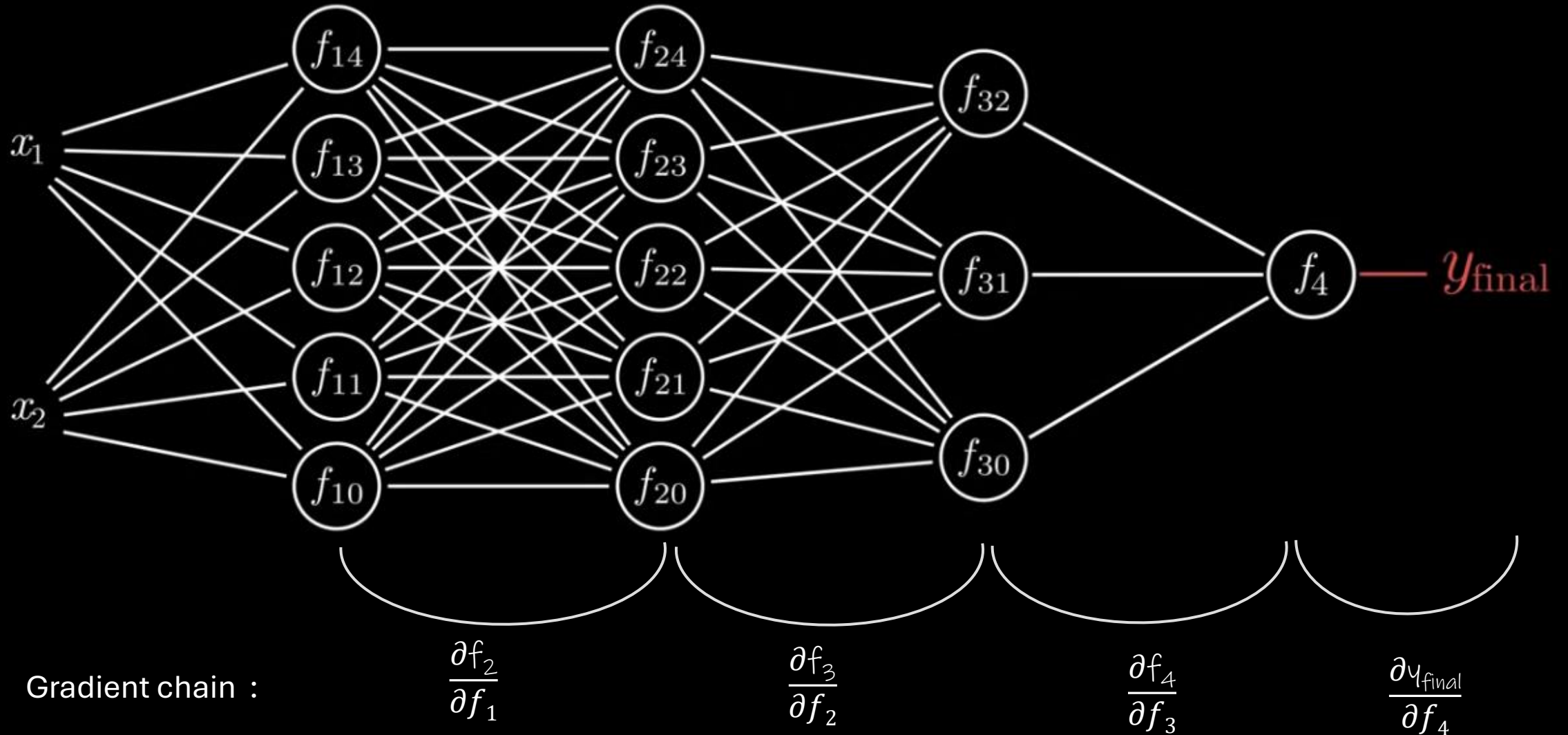


The Multilayer perceptron (1986)

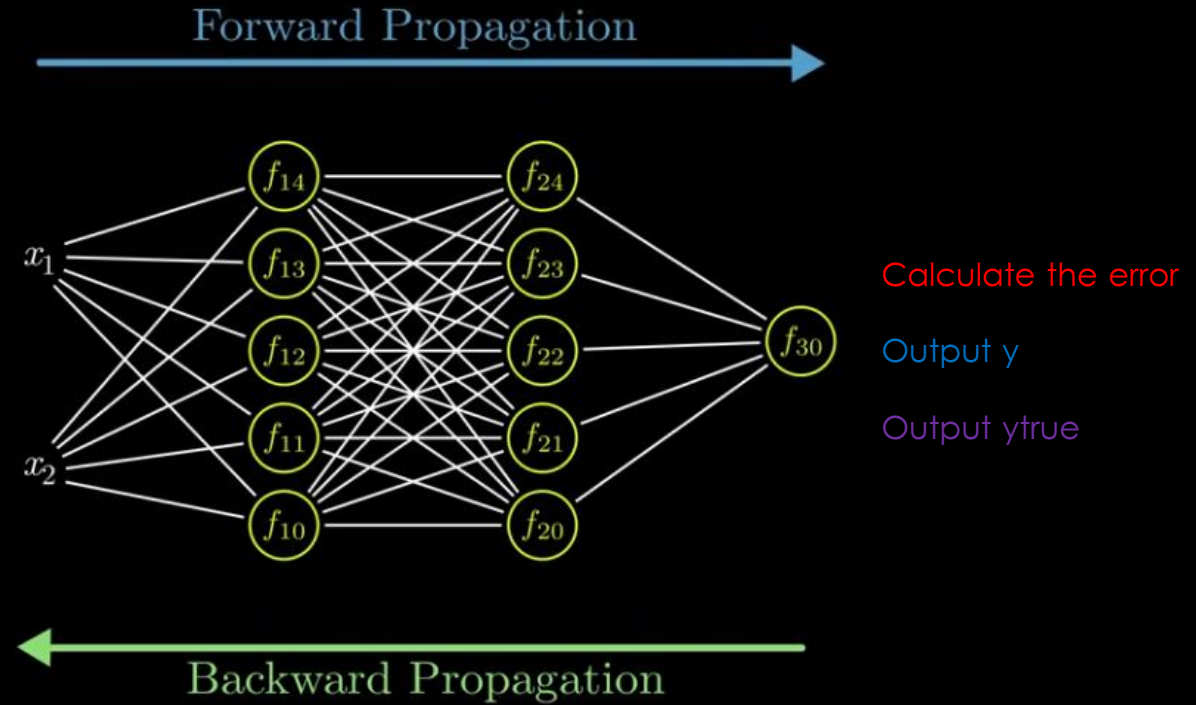


The back-propagation

How the output of the ANN varies as a function of the parameter (W,b) of each layer.



- 
1. Forward Propagation
 2. Cost Function
 3. Backward Propagation
 4. Gradient Descent



1986 : The Multilayer Perceptron (MLP)

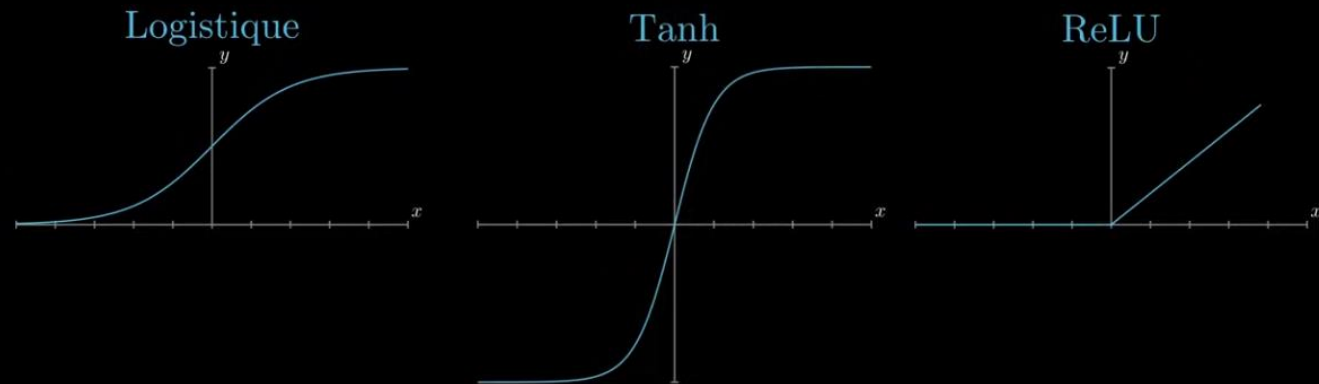


Geoffrey Hinton

1986 : The Multilayer Perceptron (MLP)



Geoffrey Hinton



1986 : The Multilayer
Perceptron (MLP)

1990 : Convolutional NN
– LeNet



Geoffrey Hinton



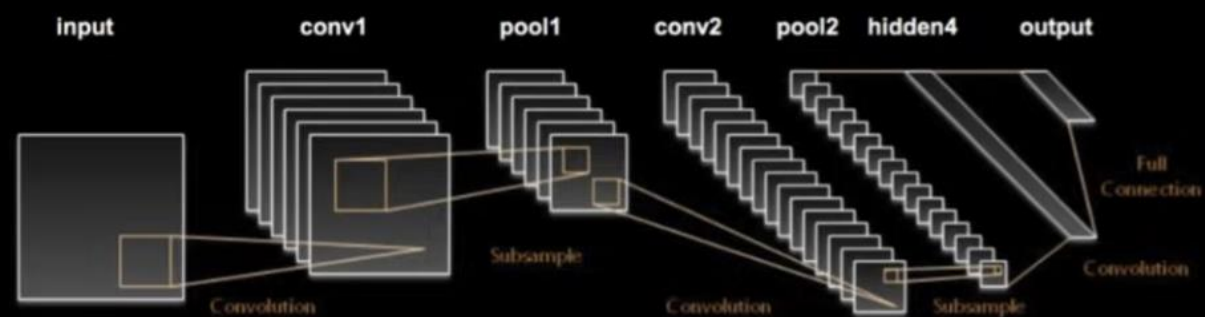
Yann LeCun

1990 : Convolutional NN

- LeNet



Yann LeCun

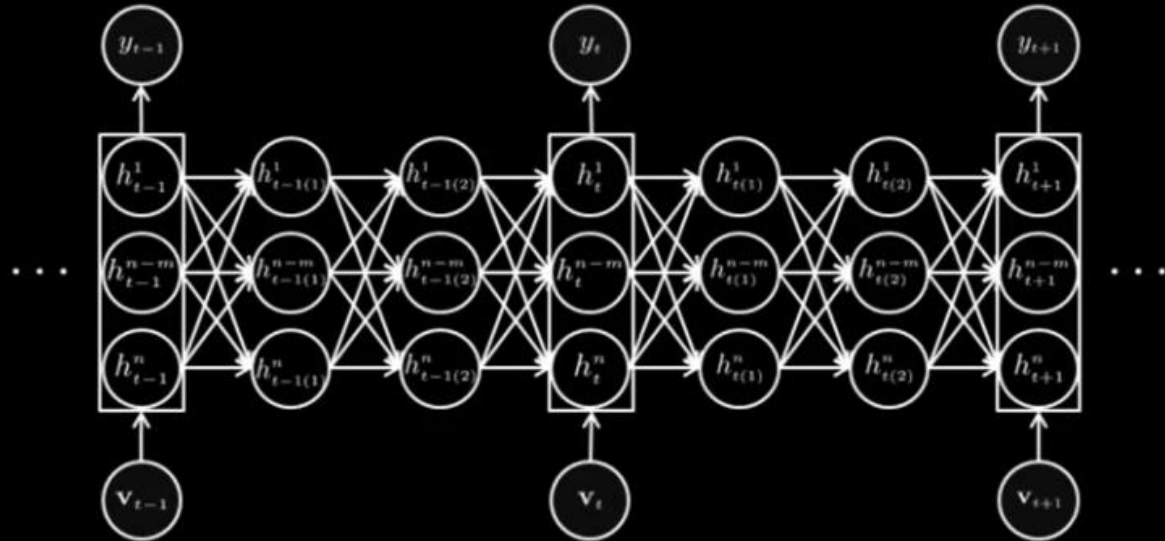


1990 : Convolutional NN
- LeNet



Yann LeCun

1997 : LSTM



2012 : ImageNet – AlexNet



Geoffrey Hinton

